

Jenga

Décrire, construire, livrer
un projet C++ sur douze plateformes

Le système de construction de Nkentseu — version 2.4.0

Table des matières

I	Prendre Jenga en main	1
1	Ce que Jenga est, et ce qu'il n'est pas	3
1.1	La phrase qui décide de tout.	3
1.2	Ce que Jenga fait.	4
1.3	Ce que Jenga n'est pas.	4
1.4	Le vocabulaire, en cinq mots.	5
1.5	Ce que vous saurez à la fin de ce livre	5
2	Où le trouver, et comment l'installer	7
2.1	Ce qu'il faut avoir avant	7
2.2	Voie 1 — le paquet installé.	7
2.3	Voie 2 — depuis les sources, en mode éditable	8
2.4	Voie 3 — embarqué dans NKCode	8
2.5	Vérifier ce qu'on a vraiment	9
2.6	Le module d'aide à l'édition	9
3	Un projet de A à Z, en cinq commandes	12
3.1	Commande 1 — créer l'espace de travail	12
3.2	Commande 2 — créer le projet	13
3.3	Commande 3 — regarder ce que Jenga a compris.	14
3.4	Commande 4 — construire	15
3.5	Commande 5 — lancer.	16
3.6	Le récapitulatif, en cinq lignes	16
II	Écrire un .jenga	18
4	Un .jenga est du Python.	20
4.1	Le squelette obligatoire	20
4.2	Ce que le Python vous donne, et qu'aucun format ne donnerait.	20
4.2.1	Une boucle	20
4.2.2	Une condition sur l'environnement	21
4.2.3	Une fonction à vous	21
4.3	Les erreurs sont des erreurs Python	22
4.4	Le répertoire courant change pendant le chargement	22
4.5	include : un seul workspace, plusieurs fichiers.	23
5	Décrire un projet : sources et sorties	25
5.1	Ce qu'un projet EST : son genre	25
5.2	Le langage et son dialecte	25
5.3	Où le projet vit : location	26
5.4	Quels fichiers lui appartiennent	26
5.5	Ce qu'il produit, et où	27
5.6	Ce qu'on ne déclare pas	27
5.7	Les métadonnées de l'application	28

6	Les filtres : une description, douze plateformes.	30
6.1	La forme	30
6.2	Les huit familles de conditions	30
6.3	Combiner	31
6.4	Les options : vos propres conditions	32
6.5	Le piège du filtre qui n'attrape rien	32
6.6	Ce qu'un filtre ne peut pas faire	33
7	Les dépendances	35
7.1	Dépendre d'un autre projet	35
7.2	Lier une bibliothèque	36
7.3	Dépendre de fichiers	36
7.4	Les chemins d'inclusion	37
7.5	Ce que Nkntseu ajoute, et qui n'est PAS de Jenga	37
	III Ce que Jenga fait de votre description	40
8	Du fichier au graphe : le chargement.	42
8.1	Les six temps du chargement	42
8.2	Les projets naissent en s'exécutant.	43
8.3	Le post-traitement	43
8.4	Que fait Jenga quand ça rate.	43
8.5	Le namespace propagé	44
8.6	Le fichier d'entrée n'est pas toujours celui qu'on croit	44
9	La construction : ordre, fraîcheur, parallélisme	46
9.1	L'ordre	46
9.2	Ce qui est recompilé, et ce qui ne l'est pas.	46
9.3	La décision se prend sur les DATES.	47
9.4	Le parallélisme	49
9.5	Nettoyer	49
	IV Les cibles	52
10	Le bureau : Windows, Linux, macOS	54
10.1	Les chaînes d'outils disponibles	54
10.2	Windows	54
10.3	Linux, et ses quatre backends	55
10.4	macOS	56
10.5	Debug et Release	57
11	Le mobile : Android, iOS, HarmonyOS	59
11.1	Android	59
11.2	iOS	60
11.3	HarmonyOS	61
11.4	Le point commun des trois.	62

12	Le Web, et ce qui est déclaré sans être éprouvé.	64
12.1	Le Web.	64
12.2	Ce qui est déclaré, et ce qui est éprouvé.	65
12.3	Les consoles	66
12.4	Choisir sa cible en ligne de commande	66
V	Au-delà de la construction	68
13	Lancer, tester, déboguer, surveiller	70
13.1	run — lancer sans connaître le chemin.	70
13.2	test — et ce qu'il répond quand il n'y a rien	70
13.3	gdb — déboguer	72
13.4	watch — reconstruire à chaque enregistrement	72
13.5	clean et rebuild.	72
14	Livrer : emballer, signer, déployer.	75
14.1	package	75
14.2	keygen et sign	76
14.3	deploy et publish	76
14.4	Vérifier un paquet, et non la commande	77
14.5	Ce qui accompagne un paquet.	77
15	Inspecter, générer, brancher son éditeur.	79
15.1	info — ce que Jenga a compris	79
15.2	compile-flags — les drapeaux réels.	79
15.3	ide-setup — brancher son éditeur	80
15.4	gen — produire un autre système de construction.	81
15.5	Les autres commandes.	81
VI	Diagnostiquer	84
16	Quand ça rate : lire ce que Jenga dit	86
16.1	Trois étages, trois familles d'échec	86
16.2	L'échec de chargement.	86
16.3	L'échec de compilation.	87
16.4	L'échec d'édition de liens	87
16.5	L'ordre de diagnostic.	88
16.6	Les messages qui ne sont pas des erreurs	88
17	Référence : les commandes et le vocabulaire	91
17.1	Les vingt-sept commandes.	91
17.2	Le vocabulaire, par famille	92
17.3	Les variables de chemin	94
17.4	Les préfixes de filtre	94
17.5	Les onze chaînes d'outils.	94

PARTIE I

Prendre Jenga en main

Trois chapitres, et vous construisez

Cette partie ne suppose rien. Elle dit ce que Jenga est — et ce qu'il n'est pas, ce qui évite autant de malentendus —, où le trouver, et comment aller d'un dossier vide à un programme qui s'exécute.

Le troisième chapitre est une **démonstration complète** : cinq commandes, et toutes les sorties qu'il montre sont des captures réelles. Vous pouvez les retaper à l'identique.

Une seule idée est à retenir avant d'aller plus loin, et elle gouverne tout le livre : **un fichier .jenga est du Python, et Jenga l'exécute**. Ce n'est pas une curiosité d'implémentation ; c'est ce qui explique aussi bien sa souplesse que ses pièges.

Ce que Jenga est, et ce qu'il n'est pas

Jenga construit des projets C et C++. Vous décrivez ce que vous voulez obtenir ; il détermine dans quel ordre compiler, avec quel compilateur, quels drapeaux, et il produit l'exécutable, la bibliothèque, l'APK ou la page Web.

Ce chapitre dit ce qu'il fait, ce qu'il ne fait pas, et surtout *par quel mécanisme* — parce que ce mécanisme explique presque tout le reste du livre.

1.1 La phrase qui décide de tout

Ce qu'il faut retenir

Un fichier `.jenga` est du Python, et Jenga l'EXÉCUTE.

Ce n'est pas une métaphore. Voici la ligne, dans le chargeur :

`Jenga/Core/Loader.py:268`

```
1 exec(filePath.read_text(encoding='utf-8-sig'), globals_dict)
```

Votre description de projet n'est pas analysée comme un format de données : elle est *lancée*. Les appels que vous écrivez — `project()`, `files()`, `links()` — sont de vraies fonctions Python qui remplissent des objets en mémoire.

Trois conséquences immédiates, et vous les emploierez dès le chapitre 3 :

Vous pouvez	Parce que
mettre une boucle <code>for</code> dans un <code>.jenga</code>	c'est du Python
lire une variable d'environnement	c'est du Python
appeler une fonction que vous écrivez	c'est du Python
importer un module	c'est du Python

Le piège

Et la contrepartie, qu'il vaut mieux connaître tout de suite : une erreur dans votre `.jenga` est une erreur *Python*, avec sa pile d'appels. Une virgule oubliée ne donne pas « fichier de projet invalide » mais `SyntaxError`.

Un `.jenga` peut aussi faire n'importe quoi — effacer un dossier, appeler le réseau. Il a tous les droits de l'interpréteur. Ne lancez pas un `.jenga` que vous n'avez pas lu, exactement comme vous ne lanceriez pas un script inconnu.

1.2 Ce que Jenga fait

Il s'occupe de	Concrètement
trouver les sources	<code>files(["src/**/*.cpp"])</code>
choisir le compilateur	selon la plateforme visée
ordonner les projets	d'après les dépendances déclarées
ne recompiler que le nécessaire	voir le chapitre 10 — et son piège
produire l'artefact final	<code>.exe</code> , <code>.a</code> , <code>.apk</code> , <code>.wasm...</code>
empaqueter et signer	<code>jenga package</code> , <code>jenga sign</code>
lancer, tester, déboguer	<code>run</code> , <code>test</code> , <code>gdb</code>
configurer votre éditeur	<code>ide-setup</code> , <code>compile-flags</code>

Ce qu'il faut retenir

Douze systèmes cibles sont déclarés, et le livre distingue soigneusement ceux qui sont *éprouvés* de ceux qui sont seulement *déclarés* — c'est l'objet du chapitre 15.

Jenga/Core/Api.py:66-83

```
1 class TargetOS(Enum):
2     WINDOWS LINUX MACOS ANDROID IOS TVOS WATCHOS IPADOS
3     VISIONOS WEB PS4 PS5 XBOX_ONE XBOX_SERIES SWITCH
4     HARMONYOS FREEBSD OPENBSD
```

1.3 Ce que Jenga n'est pas

Ce qu'il faut retenir

Ce n'est pas un gestionnaire de paquets. Il ne télécharge pas vos dépendances depuis un dépôt public. Une bibliothèque tierce est un dossier que vous avez, et vous lui écrivez un `.jenga` — ou vous la liez par `links()`.

Ce n'est pas un générateur de projets, et c'est la différence avec CMake. CMake produit un `Makefile` ou une solution Visual Studio, qui construisent ensuite. Jenga **construit lui-même** : il appelle le compilateur. (Il *peut* générer un projet, par `jenga gen`, mais c'est un service annexe, pas son fonctionnement.)

Ce n'est pas un système de tâches générique. Il ne remplace ni `make` ni un script : il connaît la compilation, l'édition de liens et l'empaquetage, et rien d'autre.

Le piège

Il ne devine pas votre bibliothèque système. Écrire `links(["X11"])` suppose que `libX11` est installée et trouvable. Jenga passe le drapeau à l'éditeur de liens ; c'est ce dernier qui échoue, et son message ne nomme pas Jenga.

C'est la source la plus fréquente d'un « Jenga ne marche pas » qui n'est pas un défaut de Jenga.

1.4 Le vocabulaire, en cinq mots

Mot	Ce qu'il désigne
<i>workspace</i>	l'espace de travail : le fichier racine, et tout ce qu'il inclut
<i>project</i>	une unité qui produit un artefact : un exe, une lib, un test
<i>configuration</i>	Debug, Release... — un jeu de réglages
<i>toolchain</i>	une chaîne d'outils : compilateur, éditeur de liens, archiveur
<i>filter</i>	une condition sous laquelle certains réglages s'appliquent

Ce qu'il faut retenir

Un workspace contient des projets ; un projet ne contient rien. C'est la seule hiérarchie. Il n'y a pas de sous-projet, pas de groupe : un projet dépend d'un autre, et c'est ainsi qu'on structure.

1.5 Ce que vous saurez à la fin de ce livre

Partie	Ce qu'elle donne
--------	------------------

- | | |
|-----|--|
| I | installer Jenga, le trouver, construire un projet de A à Z |
| II | écrire un <code>.jenga</code> : sources, sorties, filtres, dépendances |
| III | comprendre ce que Jenga fait de votre description |
| IV | viser une plateforme — et savoir laquelle est éprouvée |
| V | lancer, tester, emballer, signer, déployer |
| VI | lire un échec, éviter les pièges, écrire une chaîne d'outils |

Ce qu'il faut retenir

Tout ce que ce livre affirme a été mesuré sur la version 2.4.0, et les sorties de terminal qu'il cite sont des *captures réelles* — pas des exemples recopiés. Quand une chose n'a pas pu être vérifiée ici, il le dit.

Ce chapitre en bref

Jenga construit du C et du C++, et le fait lui-même plutôt que de générer un autre système de construction. Sa décision fondatrice est qu'un fichier `.jenga` est **du Python exécuté** : vous y disposez d'un langage complet, et vous en héritez aussi les erreurs. Il n'est ni gestionnaire de paquets, ni générateur de projets, ni ordonnanceur de tâches. Un workspace contient des projets, et c'est toute la hiérarchie.

Questions de cours

1. Qu'est-ce qu'un fichier `.jenga`, techniquement ?
2. Citez deux choses que cela vous autorise, et une qu'il faut craindre.
3. En quoi Jenga diffère-t-il de CMake ?
4. Que se passe-t-il si vous écrivez `links(["X11"])` sans avoir la bibliothèque ?

5. Quelle est la seule hiérarchie de Jenga ?

Exercice pratique

Ouvrez un .jenga réel du dépôt — par exemple `Applications/NkDames/NkDames.jenga` — et relevez :

1. les constructions qui sont du *Python* et non du vocabulaire Jenga (`import`, `def`, `if`, `os.getenv`);
2. le nombre de blocs `with filter(...)`;
3. ce que le fichier ferait de différent si vous supprimiez la ligne `files([...])`.

Répondez à la troisième *avant* de l'essayer.

Exercice de démonstration

Prouvez qu'un .jenga est bien exécuté, et non analysé.

Ajoutez, au tout début d'un .jenga de votre choix, la ligne :

```
1 print("JE SUIS EXECUTE, et il est", __import__("datetime").datetime.now())
```

Lancez `jenga info`. La ligne doit apparaître, avec l'heure courante.

Ce que la démonstration établit, et c'est plus que « ça s'affiche » : l'heure *change* d'une exécution à l'autre. Un format de données ne peut pas produire une valeur qui change; seul du code exécuté le peut.

Retirez ensuite la ligne.  Ne la laissez pas : un `print` dans un .jenga s'exécute à *chaque* commande, y compris celles dont vous lisez la sortie avec un script.

Exercice de logique

Un collègue vous dit : « Jenga est cassé, mon projet ne compile pas, il me sort une erreur Python ».

1. Quelles sont les deux familles d'erreur possibles, et laquelle cette description désigne-t-elle ?
2. Comment distingueriez-vous, en une commande, une erreur *dans sa description* d'une erreur *dans son code C++* ?
3. Il ajoute : « et ça marchait hier ». Qu'est-ce que cette information élimine, et qu'est-ce qu'elle n'élimine pas ?

Où le trouver, et comment l'installer

Jenga s'obtient de trois façons, et elles ne servent pas le même usage. Ce chapitre les donne toutes les trois, avec la façon de vérifier laquelle vous avez réellement — ce qui n'est pas la même question.

2.1 Ce qu'il faut avoir avant

Il faut	Version	Pourquoi
Python	≥ 3.8	Jenga est écrit en Python
un compilateur	clang, gcc ou MSVC	Jenga l'appelle, il ne le remplace pas
watchdog	≥ 2.1	seulement pour jenga watch
colorama	$\geq 0.4.4$	couleurs sous Windows

Ce qu'il faut retenir

Les deux dernières lignes sont facultatives, et le fichier de dépendances le dit lui-même : « optional but recommended for 'watch' command ». Jenga construit sans elles.

2.2 Voie 1 — le paquet installé

C'est la voie normale. Jenga se distribue comme une roue Python (*wheel*); l'installer place une commande jenga dans votre PATH :

pyproject.toml:37-38

```
1 [project.scripts]
2 jenga = "Jenga.Jenga:main"
```

Installer

```
1 pip install jenga-2.4.0-py3-none-any.whl --force-reinstall
```

Ce qu'il faut retenir

--force-reinstall n'est pas une précaution superstitieuse. Sans lui, pip peut considérer qu'une version portant le même numéro est déjà là et ne rien faire — en affichant un message de succès. Vous croiriez avoir installé la neuve.

2.3 Voie 2 — depuis les sources, en mode éditable

Pour travailler *sur* Jenga, on l’installe en mode éditable : la commande pointe alors vers l’arbre de sources, et une modification prend effet sans réinstaller.

Dans le depot de Jenga

```
1 python -m pip install -e . --no-build-isolation
```

Le dépôt fournit un script qui fait tout : nettoyage, réinstallation éditable, construction de la roue et des archives d’exemples.

Dans le depot de Jenga

```
1 ./cri.sh          # dev + distribution
2 ./cri.sh --tests  # ... et la suite pytest
```

Le piège

Ce script est destructeur, et dans cet ordre : il fait `rm -rf build dist` puis `pip uninstall Jenga`, et *seulement ensuite* réinstalle. Un échec en cours de route vous laisse sans commande `jenga`.

La parade tient en une ligne, et elle se pose AVANT : copiez `dist/*.whl` ailleurs. La panne devient réparable par `pip install <roue sauvegardée> --force-reinstall`. Un filet posé après l’accident n’est pas un filet.

2.4 Voie 3 — embarqué dans NKCode

NKCode contient sa propre copie de Jenga. C’est ce qui rend les boutons *Construire* et *Exécuter* de l’éditeur autonomes : ils ne dépendent pas de ce que vous avez installé.

Ce qu’il faut retenir

Le mécanisme est explicite dans le descripteur de NKCode : il **extraie la roue officielle** — environ 1 Mo — dans `Build/_jenga-embed/`, et embarque ce contenu.

Applications/NKCode/NKCode.jenga

```
1 stage = _norm(os.path.join(_REPO_ROOT, "Build", "_jenga-embed"))
2 # Re-extraie a chaque evaluation : un wheel plus recent doit gagner,
3 # sinon on embarquerait silencieusement une version perimee.
```

Il cherche la roue dans `dist/` du dépôt Jenga, en remontant plusieurs niveaux, et la variable d’environnement `NKCODE_JENGA_SRC` permet de lui indiquer un autre emplacement.

Deux versions dans la même session

NKCode peut construire avec une version et votre terminal avec une autre. Vous auriez alors **deux systèmes de construction silencieusement différents**, et un défaut d’outil se chercherait dans le code.

Le descripteur compare donc les deux et le *dit* :

Applications/NKCode/NKCode.jenga

```
1 [NKCode] Jenga embarque depuis le wheel : jenga-2.4.0-py3-none-any.whl
2 [NKCode] ATTENTION : le wheel embarque est en 2.2.0 alors que le jenga du
3 [NKCode] systeme est en 2.4.0. NKCode construira donc AUTREMENT que votre
4 [NKCode] terminal. Remede : ./cri.sh dans le depot Jenga.
```

Le défaut qui rendait cette comparaison muette

Ce tri de roues se faisait autrefois sur le *nom*, donc en texte. En ordre lexicographique, **2.9.0 est supérieur à 2.10.0** : le jour du passage à deux chiffres, NKCode aurait choisi la plus ancienne, en silence.

Il compare désormais des *tuples d'entiers*. *Comparer des versions en texte marche jusqu'à 9 et casse à 10* — et la panne se présente comme «l'outil annonce la mauvaise version», à des lieues de sa cause.

2.5 Vérifier ce qu'on a vraiment

Capture réelle

```
1 $ jenga --version
2
3           Multi-platform C/C++ Build System v2.4.0
4
5 Jenga version 2.4.0
```

Le sujet mesuré n'est pas toujours celui qu'on croit

pip show jenga et jenga --version peuvent ne pas parler du même programme. Un poste peut porter plusieurs Python : pip appartient à l'un, et le lanceur jenga de votre PATH peut venir des Scripts d'un autre.

Mesuré sur le poste de développement : python résout vers un Python 3.14, tandis que jenga vient des Scripts d'un Python 3.13.

Le sujet à mesurer est jenga --version — la commande que vous tapez — jamais pip show seul.

2.6 Le module d'aide à l'édition

Vous verrez apparaître un dossier `.jenga-typings/` à côté de vos descripteurs. Il contient deux fichiers, et le premier avoue sa nature :

.jenga-typings/jengaconfig.py

```
1 # Module no-op genere par Jenga. `from jengaconfig import *` est resolu
2 # par jengaconfig.pyi cote editeur ; au runtime les symboles viennent de
3 # la propagation useconfig(). Ne rien mettre ici.
```

Ce qu'il faut retenir

`from jengaconfig import *` n'importe rien. C'est une ligne *pour l'éditeur* : elle donne l'auto-complétion via le fichier `.pyi`. À l'exécution, les symboles arrivent par la propagation de `useconfig()`.

C'est un cas rare et honnête d'une ligne qui ne sert qu'à l'outillage — et qu'il faut savoir reconnaître, sans quoi on la cherche dans le mécanisme.

Ce chapitre en bref

Trois voies : la roue installée pour s'en servir, le mode éditable pour travailler dessus, et la copie embarquée dans `NKCode` pour que l'éditeur soit autonome. La troisième peut diverger des deux autres, et `NKCode` le signale — parce que deux systèmes de construction dans une même session font chercher un défaut d'outil dans le code. Vérifiez toujours par `jenga --version`, qui interroge la commande réelle, et non par `pip show`, qui peut parler d'un autre Python. Enfin, `jengaconfig` ne sert qu'à votre éditeur.

Questions de cours

1. Quelles sont les trois façons d'avoir Jenga, et à quoi sert chacune ?
2. Pourquoi `--force-reinstall` lors de l'installation d'une roue ?
3. Dans quel ordre `cri.sh` désinstalle-t-il et réinstalle-t-il, et qu'est-ce que cela implique ?
4. Pourquoi `NKCode` compare-t-il sa version embarquée à celle du système ?
5. Que fait `from jengaconfig import *` à l'exécution ?

Exercice pratique

Installez Jenga, puis établissez — par des commandes, pas par déduction — les quatre faits suivants :

1. la version que jenga annonce ;
2. la version que `pip` croit avoir installée ;
3. le chemin exact du lanceur jenga (`where` ou `which`) ;
4. l'interpréteur Python auquel ce lanceur est rattaché.

Si les points 1 et 2 diffèrent, vous avez deux Python. Notez lequel gagne.

Exercice de démonstration

Prouvez que le mode éditable est bien *éditable*.

Installez Jenga par `pip install -e .`, puis ajoutez une ligne `print("MARQUE")` au tout début de `Jenga/Jenga.py`. Relancez `jenga --version` : la marque doit apparaître, **sans réinstaller**.

Retirez la ligne, relancez : elle disparaît.

Ce que la démonstration établit : la commande lit l'arbre de sources à chaque exécution. Refaites ensuite le même essai après une installation par *roue* — la marque n'apparaîtra pas, parce que le code exécuté est une copie. C'est la paire qui prouve, pas le premier essai seul.

Exercice de logique

Vous corrigez un défaut dans Jenga, vous relancez votre construction, et rien ne change.

1. Énumérez au moins quatre causes possibles, de la plus banale à la plus retorse.
2. Pour chacune, donnez une commande dont le résultat *diffère* selon que la cause est celle-là ou une autre.
3. L'une de ces causes ne se voit que si l'on regarde *quel* programme a construit — laquelle, et pourquoi les autres vérifications passent-elles à côté ?

Un projet de A à Z, en cinq commandes

Ce chapitre part d'un dossier vide et arrive à un programme qui s'exécute. Cinq commandes, et rien d'autre.

Toutes les sorties de ce chapitre sont des captures réelles, prises en enchaînant ces commandes. Rien n'est recopié de mémoire.

3.1 Commande 1 — créer l'espace de travail

```
1 jenga workspace Bonjour --no-interactive
```

Trois fichiers apparaissent :

Ce que la commande a produit

```
1 Bonjour/.gitignore
2 Bonjour/Bonjour.jenga
3 Bonjour/pyrightconfig.json
```

Ce qu'il faut retenir

Un des trois ne sert qu'à votre éditeur. `pyrightconfig.json` fait que l'auto-complétion connaît le vocabulaire Jenga dans vos `.jenga` — qui sont du Python, donc analysables. Il n'appartient à aucun éditeur en particulier : tout client LSP le lit, NKCode compris. Le seul fichier qui compte pour la construction est `Bonjour.jenga`.

Le piège

Jusqu'au 2 septembre 2026, il y en avait **quatre** — un `.vscode/settings.json` apparaissait aussi, pour un éditeur que vous n'aviez peut-être jamais ouvert. La fonction de détection finissait par « si rien n'est détecté, on assume VSCode ».

Ce n'est pas le dossier de trop qui était grave, c'est qu'il **se confirmait lui-même** : le `.vscode/` ainsi créé était *ensuite* détecté par le test qui décide d'écrire, donc une *supposition* du premier passage devenait un *fait* au second, définitivement — et plus personne ne pouvait répondre à « qui a voulu ce dossier? ».

Jenga n'écrit désormais que pour un éditeur **dont la trace existe** : `.vscode/`, `.idea/`, `.nkcode/`... Sans trace, seul `pyrightconfig.json` sort. Un utilisateur de VSCode qui part de zéro le demande une fois, et c'est tout :

```
1 jenga ide-setup --editor vscode
```

Un opt-in explicite vaut mieux qu'un dossier apparu tout seul.

Et voici ce qu'il contient — lisez-le, il tient en dix lignes utiles :

Bonjour/Bonjour.jenga, genere

```
1 from Jenga import *
2
3 with workspace("Bonjour"):
4     configurations(['Debug', 'Release'])
5     targetoses([TargetOS.WINDOWS, TargetOS.LINUX, TargetOS.MACOS])
6     targetarchs([TargetArch.X86_64])
7
8     # Default toolchain (auto-detected)
9     # usetoolchain("host-gcc")
10
11     # Add your projects here
12     # with project("MyApp"):
13     #     consoleapp()
14     #     language("C++")
15     #     files(["src/**/*.cpp"])
```

Ce qu'il faut retenir

Aucun projet encore, et c'est délibéré. Un workspace décrit le *cadre* : quelles configurations existent, quels systèmes on vise, quelles architectures. Les projets viennent ensuite — et le gabarit vous montre déjà leur forme, en commentaire.

3.2 Commande 2 — créer le projet

```
1 jenga project Bonjour --kind console --no-interactive
```

Le nom du *kind* n'est pas celui de la fonction

En ligne de commande, c'est `console`. Dans le `.jenga`, la fonction s'appelle `consoleapp()`. Se tromper donne, textuellement :

Capture réelle, code de sortie 2

```
1 jenga project: error: argument --kind: invalid choice: 'consoleapp'
2 (choose from console, windowed, static, shared, test)
```

Le message est bon : il énumère les cinq valeurs admises. C'est le genre d'erreur qui se résout en la lisant — et qui fait perdre dix minutes quand on ne la lit pas.

La commande crée la source et **modifie le workspace** :

Bonjour/Bonjour.jenga, apres 'jenga project'

```
1 # Project: Bonjour
2 with project("Bonjour"):
3     consoleapp()
4     language("C++")
5     location("Bonjour")
6     files(["src/**/*.cpp", "include/**/*.hpp"])
```

Bonjour/Bonjour/src/main.cpp, genere

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello from Bonjour!" << std::endl;
5     return 0;
6 }
```

Ce qu'il faut retenir

Cinq lignes décrivent entièrement un programme. Son genre, son langage, où il vit, et quels fichiers lui appartiennent. Tout le reste — compilateur, drapeaux, dossier de sortie — a une valeur par défaut que vous ne redéfinirez que si elle ne vous convient pas.

3.3 Commande 3 — regarder ce que Jenga a compris

jenga info – capture réelle, abrégée

```
1 ===== Jenga Workspace: Bonjour =====
2
3 Configurations: Debug, Release
4 Platforms: Windows
5 Target OSes: Windows, Linux, macOS
6 Target Architectures: x86_64
7
8 Projects
9 -----
10 Name      Kind      Language  Test  External
11 =====
12 Bonjour   ConsoleApp  C++      No    No
13
14 Available Toolchains
15 -----
16 Name      Family      Target OS  Arch  Env
17 =====
18 host-clang clang       Windows   x86_64 mingw
19 host-gcc   gcc         Windows   x86_64 mingw
20 msvc      msvc        Windows   x86_64 msvc
21 clang-mingw clang       Windows   x86_64 mingw
22 mingw     gcc         Windows   x86_64 mingw
23 clang-cross-linux clang      Linux     x86_64 gnu
24 android-ndk android-ndk Android   arm64  android
25 emscripten emscripten  Web       wasm32
```

Ce qu'il faut retenir

jenga info est la première commande à taper quand quelque chose surprend. Elle montre ce que Jenga a *compris* de votre description — pas ce que vous croyez avoir écrit. Notez la distinction entre *Platforms* : *Windows* (la machine sur laquelle on est) et *Target OSes* : *Windows*, *Linux*, *macOS* (ce que le workspace déclare viser). Confondre les deux fait chercher pourquoi « Linux n'est pas là » alors qu'il l'est. Et la table des chaînes d'outils est **ce qui est disponible ici, sur cette machine** : Android et Emscripten y figurent parce que leurs SDK sont installés. Sur une autre machine, la table est plus courte.

3.4 Commande 4 — construire

jenga build –config Release – capture réelle

```
1 Loading workspace...
2
3 Configuration: Release
4 Target:      Windows x86_64
5 Toolchain:   clang-mingw
6
7 Build Order (1 projects):
8   1. Bonjour [CONSOLE_APP]
9
10  Project: Bonjour                                Kind: CONSOLE_APP
11
12 Found 1 source file(s)
13   [1/1] Compiled: main.cpp
14 Linking...
15 Built: Build\Bin\Release-Windows\Bonjour\Bonjour.exe
16
17 Build Successful                                Time: 0.83s
18
19 BUILD COMPLETED
20 Projects Built: 1/1
21 Time:          0.83s
22 Status:        SUCCESS
```

Ce qu'il faut retenir

Quatre lignes de cette sortie méritent d'être lues à chaque fois, et ce sont celles qu'on saute :

Ligne	Ce qu'elle évite de croire à tort
Configuration: Release	que vous mesurez du Debug
Toolchain: clang-mingw	que vous compilez avec MSVC
Build Order	qu'un projet est construit alors qu'il ne l'est pas
Compiled: main.cpp	qu'un fichier a été recompilé

La dernière est la plus importante, et le chapitre 10 lui consacre une démonstration : **un build vert prouve que ce qu'il a construit compile, pas que ce que vous avez écrit a été construit.**

Le chemin de sortie se lit, il ne se devine pas

Build\Bin\Release-Windows\Bonjour\Bonjour.exe

La forme est Build/Bin/<config>-<système>/<projet>/. Elle change avec la configuration *et* avec le système — et sous Linux elle porte en plus le backend graphique, par exemple Release-Linux-xlib. Un script qui code ce chemin en dur échoue sur une construction *réussie*.

3.5 Commande 5 — lancer

jenga run --config Release – capture réelle

```
1 EXECUTION -- Bonjour.exe
2 ...\\Build\\Bin\\Release-Windows\\Bonjour\\Bonjour.exe
3
4 Application console sans terminal : ouverture d'une console dediee.
```

Ce qu'il faut retenir

jenga run retrouve l'exécutable tout seul, à partir de la configuration et de la plateforme. Vous n'avez pas à connaître le chemin — ce qui est justement pourquoi il faut le lire au moins une fois, à l'étape précédente.

Et il ouvre une console dédiée quand l'application est de type console et qu'aucun terminal n'est attaché. C'est une commodité; elle explique aussi pourquoi la sortie du programme n'apparaît pas toujours dans *votre* terminal.

3.6 Le récapitulatif, en cinq lignes

De rien à un programme qui tourne

```
1 jenga workspace Bonjour --no-interactive
2 cd Bonjour
3 jenga project Bonjour --kind console --no-interactive
4 jenga build --config Release
5 jenga run --config Release
```

Ce qu'il faut retenir

Aucune de ces commandes n'a demandé de compilateur, de chemin, ni de drapeau. C'est le contrat de Jenga : vous décrivez ce que vous voulez, il choisit comment. Le reste du livre porte sur les moments où ce choix ne vous convient pas — et sur la façon d'en reprendre la main sans perdre le reste.

Ce chapitre en bref

Cinq commandes suffisent : workspace, project, info, build, run. Le workspace décrit le cadre, le projet décrit un artefact en cinq lignes, et info montre ce que Jenga a *compris* — la première commande à taper quand quelque chose surprend. La sortie de construction porte quatre lignes qu'il faut lire à chaque fois : la configuration, la chaîne d'outils, l'ordre, et les fichiers réellement recompilés. Enfin, le chemin de sortie dépend de la configuration et du système : il se lit, il ne se code pas en dur.

Questions de cours

1. Quels fichiers jenga workspace produit-il, et lequel compte vraiment ?
2. Pourquoi --kind consoleapp échoue-t-il alors que consoleapp() existe ?
3. Quelle différence entre *Platforms* et *Target OSes* dans la sortie de info ?
4. Citez les quatre lignes de la sortie de build qu'il faut lire, et ce que chacune évite.

5. De quoi dépend le chemin de l'exécutable produit ?

Exercice pratique

Refaites les cinq commandes, dans un dossier vide, sans regarder ce chapitre.

Puis trois variantes, chacune répondant à une question :

1. construisez en Debug — où l'exécutable atterrit-il ?
2. changez le message de `main.cpp` et reconstruisez — combien de fichiers sont recompilés ?
3. lancez `jenga build` sans modifier quoi que ce soit — qu'affiche la ligne `Compiled: ?`

La troisième prépare le chapitre 10.

Exercice de démonstration

Prouvez que `jenga info` lit votre description, et non un état mis en cache.

Ajoutez un second projet à votre workspace — cinq lignes suffisent — et relancez `jenga info` sans construire. Le tableau des projets doit en montrer deux.

Retirez-le, relancez : un seul.

Ce que la démonstration établit, et c'est ce qui la rend utile : la description est relue à chaque commande. Vous pouvez donc vous fier à `info` pour vérifier une modification avant de payer une construction.

Exercice de logique

Vous lancez `jenga build`, la sortie dit SUCCESS, et pourtant votre programme se comporte comme avant.

1. Énumérez au moins quatre explications, en distinguant celles où la compilation *n'a pas eu lieu* de celles où elle a eu lieu *sans effet*.
2. Pour chacune, dites quelle ligne de la sortie de `build` la confirmerait ou l'éliminerait.
3. Une de ces explications ne se voit dans **aucune** ligne de cette sortie. Laquelle, et par quoi la prendriez-vous ?

Ce que cette partie vous laisse

Jenga construit du C et du C++, lui-même plutôt qu'en générant un autre système. Il s'obtient de trois façons — roue installée, mode éditale, copie embarquée dans NKCode — et la troisième peut diverger des autres, ce que NKCode signale. Cinq commandes suffisent pour aller de rien à un programme qui tourne : `workspace`, `project`, `info`, `build`, `run`. Et `info` est la première à taper quand quelque chose surprend : elle montre ce que Jenga a compris.

PARTIE II

Écrire un .jenga

Le vocabulaire, et ce qu'il cache

Un descripteur Jenga se lit comme une déclaration et s'exécute comme un script. Cette partie donne les deux lectures : le vocabulaire — `project`, `files`, `filter`, `dependson` — et le mécanisme Python qui le porte.

L'ordre des quatre chapitres suit celui dans lequel vous écrirez : d'abord comprendre que c'est du code, puis décrire un projet, puis le rendre portable par les filtres, puis le relier au reste.

Chaque chapitre nomme les défauts SILENCIEUX de son sujet — un appel hors bloc, un motif qui ne trouve rien, un filtre mal orthographié, une dépendance déclarée à deux endroits. Aucun de ces quatre ne produit de message d'erreur, et c'est ce qui les rend chers.

Un .jenga est du Python

Le chapitre 1 l'a affirmé ; celui-ci en tire les conséquences pratiques. Ce n'est pas une curiosité d'implémentation : c'est ce qui vous permet d'écrire une boucle là où d'autres systèmes vous demanderaient un langage de macros.

4.1 Le squelette obligatoire

Le minimum viable

```
1 from Jenga import *
2
3 with workspace("MonEspace"):
4     configurations(['Debug', 'Release'])
5
6     with project("MonProgramme"):
7         consoleapp()
8         language("C++")
9         files(["src/**/*.cpp"])
```

Ce qu'il faut retenir

with, et pas autre chose. `workspace` et `project` sont des *gestionnaires de contexte* Python — des classes avec `__enter__` et `__exit__`.

Ce que cela signifie concrètement : **tout appel écrit à l'intérieur du bloc s'applique à ce bloc.** `files()` ne prend pas de projet en paramètre ; il sait où il est parce qu'il lit une variable globale que `with project(...)` vient de poser.

Le piège

Un appel écrit HORS d'un bloc ne disparaît pas : il s'applique ailleurs. `files()` après la fin d'un `with project` s'applique au projet *précédent*, ou à rien — pas d'erreur, pas d'avertissement.

L'indentation Python n'est donc pas cosmétique ici : **elle décide à quoi un réglage appartient.** Un décalage d'un niveau change la signification sans jamais échouer.

4.2 Ce que le Python vous donne, et qu'aucun format ne donnerait

4.2.1 Une boucle

Six demos, une description

```
1 for i in range(1, 7):
2     with project("Demo%d" % i):
3         consoleapp()
4         language("C++")
```



```

5     location("Demos/Demo%d" % i)
6     files(["src/**/*.cpp"])

```

4.2.2 Une condition sur l'environnement

Le dépôt s'en sert réellement, pour ne déclarer Vulkan que là où il est câblé :

Applications/NkDames/NkDames.jenga

```

1  _VULKAN_SDK = _norm(os.getenv("VULKAN_SDK", ""))
2  _VULKAN_OPT = os.getenv("NK_ENABLE_VULKAN", "auto").strip().lower()
3  if _VULKAN_OPT in ("0", "false", "off", "no"):
4      _WANT_VULKAN = False
5  elif _VULKAN_OPT in ("1", "true", "on", "yes"):
6      _WANT_VULKAN = True
7  else:
8      _WANT_VULKAN = bool(_VULKAN_INCLUDE) or _pkg_exists("vulkan")

```

4.2.3 Une fonction à vous

Applications/NkDames/NkDames.jenga

```

1  def _pkg_exists(pkg_name):
2      if not shutil.which("pkg-config"):
3          return False
4      try:
5          return subprocess.call(["pkg-config", "--exists", pkg_name],
6                                  stdout=subprocess.DEVNULL,
7                                  stderr=subprocess.DEVNULL) == 0
8      except Exception:
9          return False

```

Ce qu'il faut retenir

Ce descripteur interroge le système pendant qu'il se charge. Il lance `pkg-config`, lit des variables d'environnement, teste des chemins.

Aucun format déclaratif ne permet cela — et c'est exactement ce dont on a besoin quand une bibliothèque est présente sur une machine et pas sur une autre.

Le revers, et il est sérieux

Une description qui interroge le système donne un résultat qui dépend du système. Deux machines, deux descriptions effectives, et le `.jenga` est pourtant identique dans le dépôt.

C'est puissant et c'est un piège : « ça construit chez moi » devient une phrase qui ne prouve rien. **Quand vous écrivez une condition, faites-la dire quelque chose** — une ligne de journal — sinon la différence entre deux machines reste invisible jusqu'à l'édition de liens.

4.3 Les erreurs sont des erreurs Python

Ce qu'il faut retenir

Le chargeur attrape et affiche :

Jenga/Core/Loader.py

```
1 except Exception as e:
2     Colored.PrintError(f"Error loading workspace: {e}")
3     if self.verbose:
4         traceback.print_exc()
```

Avec `--verbose`, vous obtenez la pile d'appels complète. Sans lui, vous n'avez que le message — souvent suffisant, parfois non. C'est le premier réflexe quand un `.jenga` refuse de se charger.

Ce que vous voyez	Ce que c'est
<code>SyntaxError</code>	une virgule, une parenthèse, une indentation
<code>NameError</code>	une fonction Jenga mal orthographiée
<code>TypeError</code>	un argument du mauvais type — souvent une chaîne au lieu d'une liste
<code>FileNotFoundError</code>	un <code>include()</code> vers un fichier absent
<code>No workspace defined</code>	aucun <code>with workspace(...)</code> n'a été exécuté

Le piège

La plupart des fonctions attendent une LISTE. `files("src/main.cpp")` — sans crochets — ne donne pas toujours une erreur : une chaîne est itérable, et Python la parcourt *caractère par caractère*.

Vous obtenez alors un projet dont les fichiers s'appellent `s`, `r`, `c`, `/...` et le message final parlera de fichiers introuvables, sans jamais nommer la cause.

4.4 Le répertoire courant change pendant le chargement

Jenga/Core/Loader.py

```
1 old_cwd = Path.cwd()
2 os.chdir(filePath.parent)
```

Ce qu'il faut retenir

Pendant l'exécution d'un `.jenga`, le répertoire courant est celui du fichier. C'est ce qui rend les chemins relatifs prévisibles : ils sont relatifs au descripteur, pas à l'endroit d'où vous avez tapé la commande.

Et c'est vrai *récurivement* : un fichier inclus s'exécute dans son dossier. Un `include("Applications/X/X.jenga")` suivi de `location(".')` dans le fichier inclus désigne bien `Applications/X`.

4.5 include : un seul workspace, plusieurs fichiers

Nkentseu.jenga, a la racine

```
1 with include("Applications/NkDames/NkDames.jenga"):
2     pass
```

Ce qu'il faut retenir

Le fichier inclus s'exécute **en ligne**, dans le workspace courant. Il ne crée pas un second workspace : il ajoute ses projets au vôtre.

Et il hérite du **namespace** du fichier racine. Le chargeur expose les variables globales du fichier d'entrée aux inclusions :

Jenga/Core/Loader.py

```
1 # Exposer le namespace LIVE du workspace racine pour que `include()` propage
2 # automatiquement aux fichiers inclus tout ce que l'utilisateur définit ici
3 # (config/fonctions/classes/imports) -> plus besoin de les re-importer dans
4 # chaque .jenga inclus.
```

Une fonction ou une constante définie *au-dessus* des inclusions est donc visible dans tous les fichiers inclus. **Au-dessus** : les inclusions s'exécutent à l'endroit où elles sont écrites, et ne voient que ce qui les précède.

Le piège

Un projet non inclus n'existe pas. `jenga build --target MonJeu` répond que la cible est inconnue, et l'on relit son `.jenga` vingt fois — qui est parfaitement juste.

La question à se poser n'est pas « qu'ai-je mal écrit dans ce fichier ? » mais « ce fichier est-il inclus ? ». `jenga info` tranche en une seconde : le projet est dans le tableau, ou il n'y est pas.

Ce chapitre en bref

Un `.jenga` est du Python exécuté, et workspace comme project sont des gestionnaires de contexte : l'indentation décide à quoi un réglage appartient, et un appel mal placé s'applique ailleurs sans rien dire. Vous disposez des boucles, des conditions et de vos propres fonctions — au prix qu'une description peut alors dépendre de la machine, ce qu'il faut rendre visible. Les erreurs sont des erreurs Python, et `--verbose` en donne la pile. Pendant le chargement, le répertoire courant est celui du fichier, récursivement. Enfin `include` exécute en ligne, dans le même workspace, en propageant le namespace de ce qui précède.

Questions de cours

1. Pourquoi `files()` n'a-t-il pas besoin qu'on lui dise de quel projet il s'agit ?
2. Que se passe-t-il si un appel est écrit hors de son bloc ?
3. Que produit `files("src/main.cpp")` sans crochets, et pourquoi le message d'erreur ne le dit-il pas ?
4. Quel est le répertoire courant pendant l'exécution d'un `.jenga` ?

5. Une fonction définie *après* un `include` est-elle visible dans le fichier inclus ? Justifiez.

Exercice pratique

Écrivez un workspace qui déclare **cinq** projets identiques à la casse du nom près, *sans copier-coller* : une boucle.

Puis ajoutez une condition : si la variable d'environnement `DEMO_LEGER` vaut 1, seuls les deux premiers sont déclarés.

Vérifiez par `jenga info` dans les deux cas — **sans construire**. C'est le contrôle le moins cher, et il suffit ici.

Exercice de démonstration

Prouvez que l'indentation change la signification, et pas seulement la présentation.

Écrivez deux projets. Dans le premier essai, placez un `defines(["A"])` à l'intérieur du second bloc ; dans le second essai, à l'extérieur, aligné sur les `with project`.

Comparez les deux par `jenga compile-flags` — qui affiche les drapeaux réels de chaque projet.

Ce que la démonstration doit établir : dans un cas la définition est attachée à un projet, dans l'autre elle a changé de propriétaire ou disparu — **sans qu'aucune des deux exécutions n'ait échoué**. C'est cette absence d'erreur qui rend le défaut coûteux.

Exercice de logique

Un descripteur construit sur votre machine et échoue sur celle d'un collègue, avec une erreur d'édition de liens qui nomme un symbole Vulkan.

1. Le fichier étant identique dans le dépôt, expliquez comment les deux descriptions peuvent différer.
2. Où, dans le fichier, chercheriez-vous en premier ? Nommez la construction Python responsable.
3. Proposez une modification qui rende la différence *visible* au chargement, avant l'édition de liens — et dites pourquoi la placer ailleurs qu'à cet endroit ne servirait à rien.

Décrire un projet : sources et sorties

Un projet se décrit par ce qu'il *est*, ce qu'il *contient* et ce qu'il *produit*. Ce chapitre couvre les trois, avec les valeurs par défaut — car la moitié du travail consiste à ne rien écrire.

5.1 Ce qu'un projet EST : son genre

Dans le .jenga	En ligne de commande	Produit
<code>consoleapp()</code>	<code>--kind console</code>	un exécutable avec console
<code>windowedapp()</code>	<code>--kind windowed</code>	un exécutable sans console
<code>staticlib()</code>	<code>--kind static</code>	une bibliothèque statique
<code>sharedlib()</code>	<code>--kind shared</code>	une bibliothèque dynamique
<code>testsuite()</code>	<code>--kind test</code>	une suite de tests

Ce qu'il faut retenir

`windowedapp()` n'est pas cosmétique sous Windows. Il change le sous-système de l'exécutable : sans lui, une fenêtre s'ouvre avec une console noire derrière elle. Et le point d'entrée attendu change avec le sous-système — c'est ce qui produit le fameux `undefined reference to WinMain` quand la déclaration et le code ne s'accordent pas.

Ce qu'il faut retenir

Le genre se déclare souvent DEUX fois, et ce n'est pas une erreur : une fois au niveau du projet, une fois dans chaque filtre de plateforme. Certaines plateformes en changent — le Web est déclaré `consoleapp()` là où le bureau est `windowedapp()`, parce que la notion de fenêtre native n'y a pas le même sens.

5.2 Le langage et son dialecte

```
1 language("C++")
2 cppdialect("C++17")
```

Fonction	Équivalent
<code>cppdialect(...)</code>	<code>cppversion(...)</code>
<code>cdialect(...)</code>	<code>cversion(...)</code>

Ce qu'il faut retenir

Les deux noms font la même chose. **Choisissez-en un et tenez-vous-y** : deux orthographes pour un même réglage rendent une recherche dans le dépôt incomplète — on cherche `cpddialect` et l'on rate les fichiers qui disent `cppversion`.

5.3 Où le projet vit : `location`

```
1 location(".")          # le projet vit dans le dossier de son .jenga
2 location("Bonjour")   # ... dans un sous-dossier
```

Ce qu'il faut retenir

Tous les chemins du projet sont relatifs à `location`, et `location` est lui-même relatif au dossier du `.jenga` — puisque le chargeur s'y place (chapitre 4).

Un descripteur d'application inclus depuis la racine écrit donc `location(".")` et parle de son propre dossier, pas de la racine.

5.4 Quels fichiers lui appartiennent

```
1 files(["src/**/*.cpp"])          # récursif
2 files(["src/*.cpp", "src/ui/*.cpp"]) # non récursif, deux dossiers
3 excludefiles(["src/vieux/**/*.cpp"])
```

Motif	Portée
*.cpp	le dossier courant seulement
**/*.cpp	le dossier et tous ses sous-dossiers

Le piège

Un motif ne dit pas ce qu'il ne trouve pas. `files(["src/**/*.cpp"])` sur un dossier vide donne un projet à zéro source — et l'échec sort à l'édition de liens, sur un symbole manquant, sans jamais mentionner que rien n'a été compilé.

La sortie de construction porte pourtant la réponse : `Found 1 source file(s)`. Lisez ce nombre ; c'est la ligne qui vous dit que votre motif a mordu.

Ce qu'il faut retenir

`excludemainfiles()` existe pour un cas précis : retirer les fichiers qui portent un `main()` quand on veut réutiliser les mêmes sources dans une bibliothèque ou une suite de tests. Deux `main` dans une même édition de liens ne pardonnent pas.

5.5 Ce qu'il produit, et où

Fonction	Ce qu'elle fixe
<code>objdir(chemin)</code>	où vont les fichiers objets
<code>targetdir(chemin)</code>	où va l'artefact final
<code>targetname(nom)</code>	le nom de l'artefact, si différent du projet

La forme employée partout dans Nkentseu

```
1 objdir("%{wks.location}/Build/Obj/%{cfg.buildcfg}-%{cfg.system}%{prj.name}")
2 targetdir("%{wks.location}/Build/Bin/%{cfg.buildcfg}-%{cfg.system}%{prj.name}")
```

Ce qu'il faut retenir

Les `%{...}` sont des variables résolues par Jenga, pas du Python :

<code>%{wks.location}</code>	le dossier du workspace
<code>%{cfg.buildcfg}</code>	Debug ou Release
<code>%{cfg.system}</code>	Windows, Linux...
<code>%{prj.name}</code>	le nom du projet

Sans la configuration et le système dans le chemin, un build Debug écraserait le Release et un build Linux écraserait le Windows. Ce n'est pas du rangement : c'est ce qui rend deux constructions simultanées possibles.

Le piège

Certaines plateformes ajoutent un suffixe. Sous Linux, les descripteurs de Nkentseu déclarent un dossier par backend graphique :

Applications/NkDames/NkDames.jenga

```
1 objdir("../Build/Obj/%{cfg.buildcfg}-%{cfg.system}-xlib/%{prj.name}")
```

D'où Release-Linux-xlib et non Release-Linux. Un script d'empaquetage qui code le second en dur échoue sur une construction **réussie** — et c'est arrivé.

5.6 Ce qu'on ne déclare pas

Ce qu'il faut retenir

La moitié d'un bon descripteur est ce qu'on n'y écrit pas. Le compilateur, le niveau d'avertissement, l'extension de l'artefact, la façon d'appeler l'éditeur de liens : Jenga les déduit du genre, du langage et de la plateforme.

N'écrivez un réglage que lorsque le défaut ne convient pas — et quand vous le faites, écrivez *pourquoi* à côté. Un drapeau sans justification survit dix ans après que sa raison a disparu.

5.7 Les métadonnées de l'application

```
1 apppublisher("Rihen Universe")
2 appversion("0.1.0")
3 licensefile("../..//LICENSE")
4 appicon("Resources/icone.png")
```

Ce qu'il faut retenir

Elles ne servent pas à la compilation mais à l'*empaquetage* : elles remplissent le manifeste Android, les propriétés de l'exécutable Windows, l'installateur.

appicon pointe un fichier qui doit exister. Déclarer une icône absente fait échouer la génération de l'APK — alors que la construction bureau, elle, ne dit rien. Le descripteur de NkDames porte d'ailleurs le commentaire, et la ligne est mise en attente jusqu'à ce que l'icône soit dessinée.

Ce chapitre en bref

Un projet déclare son genre, son langage, son dossier, ses fichiers et ses sorties — et rien de plus si les défauts conviennent. Le genre change le sous-système et donc le point d'entrée attendu ; les motifs `*` et `**` ne fouillent pas de la même façon, et la ligne `Found N source file(s)` est ce qui vous dit si le vôtre a mordu. Les chemins de sortie portent la configuration et le système parce que sans eux deux constructions s'écrasent — et certaines plateformes y ajoutent un suffixe qu'un script ne doit pas coder en dur. Enfin les métadonnées ne servent qu'à l'empaquetage, mais une icône déclarée doit exister.

Questions de cours

1. Quelle différence entre `consoleapp()` et `windowedapp()`, au-delà de la console ?
2. Que fouille `src/*.cpp` que `src/**/*.cpp` fouille aussi, et inversement ?
3. Quelle ligne de la sortie de construction dit qu'un motif a trouvé quelque chose ?
4. Pourquoi la configuration et le système figurent-ils dans le chemin de sortie ?
5. À quoi servent `apppublisher` et `appversion` ?

Exercice pratique

Reprenez le projet du chapitre 3 et faites-lui produire, tour à tour :

1. une bibliothèque statique au lieu d'un exécutable ;
2. un exécutable dont le *nom* diffère du projet ;
3. un exécutable dont les objets vont dans un dossier à vous.

Après chaque changement, **trouvez l'artefact sur le disque** avant de relancer `jenga run`. C'est la seule façon de vérifier que le réglage a été honoré et pas seulement accepté.

Exercice de démonstration

Prouvez qu'un motif qui ne trouve rien ne le dit pas.
Créez un projet dont `files()` pointe un dossier **vide**. Construisez.

Ce que la démonstration doit établir, et c'est en deux temps :

1. relevez ce qu'affiche la ligne `Found ... source file(s)`;
2. relevez le message d'échec final, et constatez qu'il parle d'un *symbole*, pas d'un motif.

Puis ajoutez un fichier et refaites la mesure. C'est la paire qui prouve que la première ligne était le seul indice utile.

Exercice de logique

Un collègue a écrit `targetdir("Build/Bin")` — sans configuration ni système.

1. Décrivez précisément ce qui se passe quand il construit en Debug, puis en Release, puis relance son programme.
2. Le symptôme qu'il rapportera n'est pas « mes builds s'écrasent ». Que dira-t-il, à votre avis, et pourquoi cette formulation envoie-t-elle chercher au mauvais endroit ?
3. Une troisième construction, pour une *autre plateforme*, aggrave le cas d'une façon nouvelle. Laquelle ?

Les filtres : une description, douze plateformes

Un filtre est une condition sous laquelle certains réglages s'appliquent. C'est le mécanisme qui permet à un seul .jenga de décrire un programme pour Windows, Linux, Android et le Web sans se dédoubler.

6.1 La forme

```
1 with filter("system:Windows"):
2     defines(["WIN32_LEAN_AND_MEAN"])
3     links(["user32", "gdi32", "opengl32"])
4
5 with filter("config:Release"):
6     defines(["NDEBUG"])
7     optimize("Speed")
8     symbols(False)
```

Ce qu'il faut retenir

Un filtre est un gestionnaire de contexte, comme **project**. Tout ce qui est écrit dedans est *étiqueté* de cette condition et rangé de côté; au moment de construire, Jenga ne retient que les étiquettes qui correspondent à la cible du jour.

Rien n'est évalué à l'écriture : `with filter("system:Linux")` sur une machine Windows ne saute pas le bloc — il l'enregistre. C'est ce qui permet de décrire douze plateformes depuis une seule.

6.2 Les huit familles de conditions

Elles sont dans l'évaluateur, et ce sont les seules :

Préfixe	Exemple	Teste
system:	system:Windows	le système visé
config:	config:Debug	la configuration
arch:	arch:arm64	l'architecture
platform:	platform:Windows-x86_64	système + architecture
action:	action:gen-cmake	la commande en cours
options:	options:linux-backend=xlib	une option déclarée
kind:	kind:StaticLib	le genre du projet
language:	language:C++	le langage
toolset:	toolset:clang	la famille de compilateur

Ce qu'il faut retenir

configurations:, configuration:, config: et cfg: sont **quatre orthographes du même test**. De même architecture: et arch:.
C'est commode et c'est un piège de recherche : chercher config: dans un dépôt rate les fichiers qui écrivent cfg:. **Fixez une orthographe par équipe.**

6.3 Combiner

Applications/NkDames/NkDames.jenga – reel

```

1 with filter("system:Windows && !options:windows-runtime=uwp && !system:XboxSeries"):
2     ...
3 with filter("system:Linux && options:linux-backend=xlib || "
4             "system:Linux && !options:linux-backend && !options:headless"):
5     ...

```

&&	les deux conditions
	l'une ou l'autre
!	la négation

Ce qu'il faut retenir

La seconde expression mérite d'être lue lentement : elle dit « backend xlib demandé explicitement, *ou* aucun backend demandé et pas de mode sans écran ». C'est ainsi qu'on écrit un **défaut** : le second membre attrape le cas où l'utilisateur n'a rien choisi. Sans lui, ne rien demander ne donnerait *aucun* backend — et l'échec sortirait à l'édition de liens.

Ce qu'il faut retenir

Une liste ou un ensemble passés à filter sont joints par && :

Jenga/Core/Api.py:597-609

```
1 if isinstance(expr, (list, tuple)):
2     parts = [str(part).strip() for part in expr if str(part).strip()]
3     return " && ".join(parts)
```

`filter(["system:Windows", "config:Release"])` vaut donc `"system:Windows && config:Release"`.  Un *ensemble* est en plus trié — pour que la clé soit stable, l'ordre d'un ensemble Python ne l'étant pas.

6.4 Les options : vos propres conditions

```
1 newoption({
2     "trigger": "linux-backend",
3     "value": "xlib",
4     "description": "Backend fenetrage sous Linux",
5     "allowed": ["xlib", "xcb", "wayland", "headless"],
6 })
```

Puis, en ligne de commande

```
1 jenga build --platform linux --linux-backend=wayland
```

Ce qu'il faut retenir

allowed n'est pas décoratif : il fait refuser une valeur inconnue *au chargement*, avec un message qui énumère les valeurs admises — exactement comme l'erreur de `--kind` du chapitre 3.

Sans cette liste, une faute de frappe donne une option silencieusement ignorée, et vous construisez pour un backend que vous n'avez pas demandé.

6.5 Le piège du filtre qui n'attrape rien

Le piège

Un filtre qui ne correspond à rien ne produit **AUCUN** message. C'est normal — c'est même son travail — mais cela signifie qu'une faute de frappe dans la condition est indiscernable d'une condition légitimement non remplie.

with `filter("system:windows")` en minuscules fonctionne, l'évaluateur compare en minuscules. Mais with `filter("systeme:Windows")` — en français — ne correspond à *aucune* famille connue, ne déclenche rien, et ne se plaint pas.

Le symptôme sera une bibliothèque manquante à l'édition de liens.

Comment on l'attrape

`jenga compile-flags` affiche les drapeaux réellement retenus pour chaque projet. Si votre `defines` n'y est pas, son filtre n'a pas mordu.

C'est le seul contrôle qui distingue « la condition est fausse aujourd'hui » de « la condition ne peut jamais être vraie » — en la testant sur une cible où elle *devrait* être vraie.

6.6 Ce qu'un filtre ne peut pas faire

Ce qu'il faut retenir

Un filtre ne crée pas de projet. Il modifie les réglages d'un projet existant. Pour ne déclarer un projet que sur certaines plateformes, employez du **Python** — un `if` autour du `with project` — puisque le fichier est exécuté.

Les deux mécanismes ont l'air interchangeables et ne le sont pas : le `if` décide *au chargement*, sur la machine courante ; le filtre décide *à la construction*, sur la cible. Un `if` sur `sys.platform` rendrait votre descripteur incapable de décrire une construction croisée.

Ce chapitre en bref

Un filtre étiquette des réglages d'une condition ; rien n'est évalué à l'écriture, ce qui permet de décrire douze plateformes depuis une seule. Neuf familles de conditions existent, certaines avec plusieurs orthographes — fixez la vôtre. Les expressions se combinent, et un second membre sert souvent à attraper le cas « rien n'a été demandé ». Vos propres options se déclarent par `newoption`, dont la liste `allowed` transforme une faute de frappe en erreur au lieu d'un silence. Enfin un filtre ne crée pas de projet : cela, c'est le rôle du Python — mais un `if` décide sur la machine courante là où un filtre décide sur la cible.

Questions de cours

1. Que fait Jenga d'un bloc `filter` dont la condition est fausse sur la machine courante ?
2. Citez cinq familles de conditions.
3. Que vaut `filter(["a", "b"])` ?
4. Pourquoi une expression de filtre comporte-t-elle souvent un second membre avec des négations ?
5. Quand faut-il un `if` Python plutôt qu'un filtre ?

Exercice pratique

Reprenez votre projet et donnez-lui trois comportements :

1. en `Debug`, une définition `MODE_BAVARD` ;
2. en `Release`, l'optimisation en vitesse et pas de symboles ;
3. une option à vous, `--couleur=` avec trois valeurs admises, qui pose une définition différente selon la valeur.

Vérifiez les trois par `jenga compile-flags`, pas par la construction : c'est plus rapide et plus précis.

Exercice de démonstration

Prouvez qu'un filtre mal orthographié se tait.

Écrivez deux filtres qui posent chacun une définition : l'un correct (`system:Windows`), l'autre

volontairement faux (systeme:Windows). Construisez sur Windows.

Ce que la démonstration doit établir, en trois relevés :

1. la construction **réussit**;
2. aucun message ne mentionne le filtre fautif;
3. `jenga compile-flags` montre la première définition et pas la seconde.



Les deux premiers points sont le cœur de la démonstration. Si vous ne relevez que le troisième, vous prouvez que l'outil sait montrer les drapeaux — pas que le défaut est silencieux.

Exercice de logique

Un projet lie x11 sur Linux. Sur la machine d'un collègue, l'édition de liens échoue en disant que x11 est introuvable.

1. Énumérez trois causes, en distinguant celles qui vivent dans le `.jenga` de celles qui vivent sur la machine.
2. Pour chacune, dites ce que montrerait `jenga compile-flags` — et pour laquelle il ne montrerait *rien de particulier*.
3. Cette dernière est la plus fréquente. Expliquez pourquoi l'outil ne peut pas la voir, et ce qui la verrait.

Les dépendances

Un projet dépend rarement de rien. Ce chapitre couvre les trois formes de dépendance — entre projets, vers des bibliothèques, vers des fichiers — et la distinction, souvent floue, entre *lier* et *dépendre*.

7.1 Dépendre d'un autre projet

```
1 dependson(["NKCore", "NKMath"])
```

Ce qu'il faut retenir

dependson fait deux choses à la fois, et c'est pour cela qu'il existe :

1. il impose un **ordre** — les dépendances se construisent avant;
2. il fait **hériter** ce que la dépendance exporte : ses chemins d'inclusion, ses définitions publiques.

C'est l'héritage qui compte le plus. Sans lui, il faudrait répéter les chemins d'inclusion de chaque bibliothèque dans chaque consommateur — et les répéter, c'est les laisser diverger.

Ce qu'il faut retenir

La sortie de construction montre l'ordre calculé, et c'est une ligne à lire :

Capture réelle

```
1 Build Order (1 projects):
2   1. Bonjour [CONSOLE_APP]
```

Sur un vrai dépôt elle en énumère des dizaines. **Si un projet n'y figure pas, il ne sera pas construit** — quelles que soient vos modifications dans ses sources.

Le piège

Un cycle de dépendances n'a pas de solution. A dépend de B qui dépend de A : aucun ordre ne satisfait les deux.

Le remède n'est jamais de « casser » l'ordre mais de **casser le cycle** : extraire dans un troisième projet ce dont les deux ont besoin. Un cycle est toujours le signe qu'une chose commune n'a pas été nommée.

7.2 Lier une bibliothèque

```
1 links(["user32", "gdi32", "opengl32"])
2 libdirs(["Externals/Libs"])
```

links	les bibliothèques à lier, sans préfixe ni extension
libdirs	où les chercher
syslibdirs	idem, mais chemins système
removelinks	retire d'une liste héritée

Ce qu'il faut retenir

`links(["X11"])` devient `-lX11`, et rien de plus. Jenga ne vérifie pas que la bibliothèque existe : il passe le drapeau, et c'est l'éditeur de liens qui échoue.

D'où la distinction qu'il faut garder en tête : `dependson` parle de *votre* dépôt, `links` parle du *système*. Le premier, Jenga le construit ; le second, il suppose qu'il est là.

Sur macOS et iOS, ce ne sont pas des bibliothèques

```
1 frameworks(["Cocoa", "QuartzCore", "Metal", "AudioToolbox"])
```

Un *framework* Apple est un dossier qui porte à la fois l'en-tête et le binaire. Il se déclare par `frameworks`, pas par `links`.



Et la liste n'est pas la même sur macOS et sur iOS. `AudioUnit` existe sur macOS et pas sur iOS, où ses symboles vivent dans `AudioToolbox`. Recopier la liste macOS donne `ld: framework AudioUnit not found` — mesuré. *Un précédent se vérifie sur SA plateforme.*

7.3 Dépendre de fichiers

```
1 dependfiles(["assets"])
2 embedresources(["Resources/police.ttf"])
```

dependfiles	copiés à côté de l'artefact
embedresources	intégrés <i>dans</i> le binaire

Ce qu'il faut retenir

La différence décide de votre portabilité. Un fichier copié doit être trouvé à l'exécution — ce qui dépend du répertoire courant, des permissions Android, et de la façon dont le Web sert ses fichiers. Un fichier intégré est toujours là.

Les jeux de Nkentsu ne déclarent *aucun* asset, et leur descripteur explique pourquoi :

Applications/NkDames/NkDames.jenga

```
1 # AUCUN dossier assets : NkDames ne charge aucun fichier a l'execution.
2 # Le plateau et les pieces sont dessines par geometrie, et la police est
3 # EMBARQUEE. C'est ce qui lui permet de demarrer a l'identique sur les six
4 # plateformes sans chaine de copie d'assets.
```

7.4 Les chemins d'inclusion

<code>includedirs</code>	vos en-têtes
<code>externalincludedirs</code>	ceux d'un tiers — avertissements réduits
<code>sysincludedirs</code>	en-têtes système
<code>removeincludedirs</code>	retire d'une liste héritée

Ce qu'il faut retenir

`externalincludedirs` n'est pas un synonyme. Il passe le chemin comme *externe* au compilateur, ce qui coupe les avertissements venant de ces en-têtes. Sans lui, compiler avec des avertissements exigeants noie vos propres messages sous ceux d'une bibliothèque que vous ne corrigerez pas. **Un avertissement qu'on ne peut pas traiter rend inutiles ceux qu'on pourrait traiter.**

Le piège

C'est la ligne qu'on oublie, et son symptôme accuse votre code. Sans `extra_includes` ou `includedirs`, vous obtenez `fatal error: 'MonEnTete.h' file not found` — une erreur qui pointe *votre* inclusion, alors que c'est le *projet* qui ne sait pas où chercher.

7.5 Ce que Nkentseu ajoute, et qui n'est PAS de Jenga

Ce qu'il faut retenir

Les descripteurs de Nkentseu appellent `nkentseudependson(...)`. Cette fonction **n'existe pas dans Jenga** : elle est définie dans le dépôt, à `config/modules.jenga`, ligne 334.

`config/modules.jenga:334`

```
1 def nkentseudependson(deps=None, selfexport=None, extra_includes=None,
2                       extra_links=None, extra_defines=None) -> None:
```

C'est une **démonstration de ce que le chapitre 4 annonçait** : puisqu'un `.jenga` est du Python, un dépôt peut se donner son propre vocabulaire. Ici, une seule fonction pose les dépendances, les chemins d'inclusion et les définitions selon une table centrale — au lieu de les répéter dans trente descripteurs.

Sachez distinguer les deux quand vous lisez un descripteur : ce qui vient de Jenga se retrouve dans la documentation de Jenga ; le reste est à vous, et c'est dans votre dépôt qu'il faut le chercher.

Le piège

Une dépendance déclarée à DEUX endroits ne se corrige pas à un seul. Dans Nkntseu, un module déclare ses dépendances dans son propre `.jenga` et dans la table centrale de `config/modules.jenga` — et c'est la table qui gouverne.

Mesuré : retirer une dépendance du seul `.jenga` d'un module ne l'a pas retirée du graphe. Le défaut s'est vu en *mesurant* le graphe construit, pas en relisant les fichiers.

Ce chapitre en bref

Trois formes de dépendance : `dependson` entre projets — qui impose l'ordre et fait hériter les inclusions; `links` vers le système — que Jenga suppose présent sans le vérifier; `dependfiles` et `embedresources` pour les fichiers, et le choix entre les deux décide de votre portabilité. Les frameworks Apple ne sont pas des bibliothèques, et leur liste diffère entre macOS et iOS. Enfin, un dépôt peut se donner son propre vocabulaire — `nkntseudependson` n'est pas de Jenga — et une dépendance déclarée à deux endroits ne se corrige pas à un seul.

Questions de cours

1. Quelles *deux* choses `dependson` fait-il, et laquelle compte le plus ?
2. Quelle différence entre `dependson` et `links` ?
3. Que devient `links(["X11"])`, et qui échoue si la bibliothèque manque ?
4. Pourquoi `externalincludedirs` plutôt que `includedirs` pour un tiers ?
5. `nkntseudependson` est-il une fonction de Jenga ? Où la trouve-t-on ?

Exercice pratique

Créez trois projets : une bibliothèque statique `Calcul`, une seconde `Affichage` qui en dépend, et un exécutable qui dépend des deux.

1. Vérifiez l'ordre dans la ligne `Build Order`.
2. Placez un en-tête dans `Calcul` et incluez-le depuis l'exécutable *sans* ajouter de `includedirs` — que se passe-t-il, et pourquoi ?
3. Créez volontairement un cycle et lisez ce que Jenga dit.

Exercice de démonstration

Prouvez que `dependson` fait hériter, et pas seulement ordonner.

Deux projets : une bibliothèque qui expose un en-tête, un exécutable qui l'inclut. Faites-le fonctionner avec `dependson`, puis **retirez le `dependson` en gardant `links`** sur la bibliothèque.

Ce que la démonstration doit établir : l'échec change de *nature*. Avec l'héritage, tout passe; sans lui, l'erreur n'est pas à l'édition de liens — où la bibliothèque est pourtant toujours donnée — mais à la *compilation*, sur l'en-tête introuvable.

Relevez les deux messages. C'est leur différence qui prouve que l'héritage porte les chemins d'inclusion, pas les symboles.

Exercice de logique

Vous retirez une dépendance du .jenga d'un module, vous reconstruisez, et le graphe la contient toujours.

1. Énumérez trois explications possibles.
2. Pour chacune, dites quelle commande la confirmerait — et laquelle ne se voit *pas* en relisant le fichier que vous venez de modifier.
3. Concluez sur la règle générale : que faut-il faire avant de croire qu'une suppression a pris effet ?

Ce que cette partie vous laisse

Vous savez décrire un projet : son genre, son langage, ses sources, ses sorties — et surtout ce qu'il ne faut *pas* écrire, la moitié du travail étant de laisser les défauts faire. Les filtres étiquettent des réglages d'une condition et permettent à une description de servir douze plateformes ; les dépendances ordonnent *et* font hériter. Enfin, puisque tout est du Python, un dépôt peut se donner son propre vocabulaire — au prix qu'un lecteur ne saura plus d'où vient un nom, s'il ne sait pas où le chercher.

PARTIE III

Ce que Jenga fait de votre description

Du fichier au binaire

Les deux chapitres qui suivent expliquent le mécanisme : comment un .jenga devient un graphe, puis comment ce graphe devient des fichiers objets et un artefact.

C'est la partie la plus utile du livre le jour où quelque chose ne se passe pas comme prévu, et elle contient la démonstration à refaire soi-même au moins une fois : celle qui montre qu'une source modifiée peut être déclarée à jour, et l'ancien binaire tourner — avec un SUCCESS à l'écran. Ce n'est pas un défaut de Jenga au sens strict : c'est la conséquence assumée d'une décision — comparer des dates plutôt que des empreintes — et il faut la connaître pour ne pas retirer un correctif juste en croyant qu'il ne sert à rien.

Du fichier au graphe : le chargement

Entre le moment où vous tapez `jenga build` et celui où le compilateur démarre, Jenga fait un travail qu'on ne voit pas. Ce chapitre le décrit, parce que la moitié des surprises viennent de là.

8.1 Les six temps du chargement

Temps	Ce qui se passe
1 trouver le fichier d'entrée	le <code>.jenga</code> de la racine
2 préparer l'environnement	namespace, chemin des <i>typings</i>
3 se placer	le répertoire courant devient celui du fichier
4 exécuter	<code>exec(...)</code> — vos projets naissent ici
5 récupérer le workspace	<code>getcurrentworkspace()</code>
6 post-traiter	résolution des chemins, vérifications

Jenga/Core/Loader.py, les lignes qui comptent

```

1 old_cwd = Path.cwd()
2 os.chdir(filePath.parent)
3 ...
4 _loadedFiles.append(filePath)
5 exec(filePath.read_text(encoding='utf-8-sig'), globals_dict)
6 workspace = Api.getcurrentworkspace()
7 if workspace is None:
8     raise RuntimeError("No workspace defined in the entry file.")
9 self._PostProcessWorkspace(workspace, filePath)

```

Ce qu'il faut retenir

utf-8-sig, et non utf-8. Le suffixe `-sig` fait ignorer la marque d'ordre d'octets que certains éditeurs Windows placent en tête de fichier.

Sans cela, un `.jenga` enregistré par le Bloc-notes échouerait sur sa première ligne avec une erreur de syntaxe portant sur un caractère invisible. C'est trois lettres, et elles évitent une demi-journée.

8.2 Les projets naissent en s'exécutant

Ce qu'il faut retenir

Il n'y a pas d'étape «analyse». Quand exec se termine, le graphe est déjà là : chaque `with project(...)` a créé un objet, chaque `files(...)` a rempli une liste. C'est pourquoi l'ordre d'écriture compte : une fonction doit être définie *avant* le `include` qui s'en sert, comme dans n'importe quel script.

Jenga/Core/Api.py – ce que fait 'with project'

```
1 class project:
2     def __init__(self, name: str):
3         ...
```

Ce qu'il faut retenir

Les fonctions du vocabulaire écrivent dans une variable globale. `files()` ne reçoit pas de projet : il lit `_currentProject`, que `with project` vient de poser. C'est simple, c'est efficace, et cela explique le piège du chapitre 4 : un appel hors bloc écrit dans le *dernier* projet, ou dans le vide.

8.3 Le post-traitement

Ce qu'il faut retenir

Après l'exécution, Jenga résout ce qui était relatif : les chemins deviennent absolus, les motifs de fichiers sont développés, les dépendances sont mises en correspondance avec des projets réels.

C'est ici que la plupart des erreurs de description apparaissent — et elles apparaissent *loin* de la ligne fautive, puisque cette ligne s'est exécutée sans se plaindre.

8.4 Que fait Jenga quand ça rate

Jenga/Core/Loader.py

```
1 except Exception as e:
2     Colored.PrintError(f"Error loading workspace: {e}")
3     if self.verbose:
4         traceback.print_exc()
5     return None
6 finally:
7     os.chdir(old_cwd)
```

Ce qu'il faut retenir

Trois choses à retenir de ces sept lignes.

Le **finally** restaure le répertoire courant même en cas d'échec. Sans lui, une erreur de chargement vous laisserait dans un autre dossier que celui d'où vous avez tapé la commande.

--verbose donne la pile complète. C'est le premier réflexe : le message seul dit *quoi*, la

pile dit où.

Le chargeur rend `None`, il ne lève pas. L'appelant doit donc tester — et c'est le genre de contrat qu'on oublie en écrivant un outil autour de Jenga.

8.5 Le namespace propagé

Jenga/Core/Loader.py

```
1 # Exposer le namespace LIVE du workspace racine pour que `include()` propage
2 # automatiquement aux fichiers inclus tout ce que l'utilisateur définit ici
3 # (config/fonctions/classes/imports) -> plus besoin de les re-importer dans
4 # chaque .jenga inclus.
5 Api._workspaceGlobals = globals_dict
```

Ce qu'il faut retenir

Ce qui est défini dans le fichier racine est visible partout. C'est ce qui permet à trente descripteurs d'application d'employer `nkentseudependson` sans jamais l'importer. Et le nettoyage est fait : `Api._workspaceGlobals = None` en fin de chargement, « évite toute fuite de namespace entre deux chargements de workspace ».

Le piège

Cette commodité a un coût de lisibilité. Un descripteur d'application appelle une fonction dont rien, dans son fichier, ne dit d'où elle vient. Ni `import`, ni définition. Quand vous lisez un `.jenga` et rencontrez un nom inconnu, le réflexe est donc : **chercher dans le fichier racine et dans ce qu'il inclut avant lui**, jamais dans la documentation de Jenga.

8.6 Le fichier d'entrée n'est pas toujours celui qu'on croit

Ce qu'il faut retenir

Jenga cherche le `.jenga` en remontant depuis le répertoire courant. Vous pouvez le forcer :

```
1 jenga info --jenga-file chemin/vers/Autre.jenga
```

Lancer une commande depuis un sous-dossier peut donc charger un workspace différent de celui que vous croyez. La sortie de `jenga info` donne l'`Entry file` en toutes lettres — c'est la ligne qui tranche.

Ce chapitre en bref

Le chargement tient en six temps, et le quatrième est une exécution : quand elle se termine, le graphe existe déjà. Les fonctions du vocabulaire écrivent dans une variable globale posée par le `with` courant — d'où l'importance de l'indentation. Le post-traitement résout ensuite les chemins et les dépendances, et c'est là que la plupart des erreurs se manifestent, loin de leur cause. Le namespace du fichier racine est propagé aux inclusions, ce qui rend un vocabulaire maison possible mais rend aussi son origine invisible. Enfin, la

ligne Entry file de jenga info dit quel workspace a réellement été chargé.

Questions de cours

1. À quel moment les projets existent-ils en mémoire ?
2. Pourquoi utf-8-sig et non utf-8 ?
3. Que fait le finally du chargeur, et qu'éviterait-on sans lui ?
4. D'où vient une fonction employée dans un .jenga sans y être importée ?
5. Quelle ligne de jenga info dit quel fichier a été chargé ?

Exercice pratique

Placez un print au début, au milieu et à la fin de votre .jenga, ainsi qu'au début d'un fichier inclus.

Lancez jenga info et notez l'ordre d'apparition. Puis déplacez l'include avant les définitions dont il dépend, et observez.

Retirez tous les print ensuite : ils s'exécutent à *chaque* commande, y compris celles dont un script lit la sortie.

Exercice de démonstration

Prouvez que l'include s'exécute *en ligne*, et non après.

Dans le fichier racine, définissez une variable MARQUE = "A". Placez un include. Après lui, changez MARQUE = "B". Dans le fichier inclus, affichez MARQUE.

Ce que la démonstration établit : le fichier inclus affiche A. Il voit donc l'état du namespace *au moment où il est inclus*, pas son état final.

Déplacez ensuite l'include après l'affectation à "B" et refaites la mesure. C'est la paire qui prouve, pas le premier essai.

Exercice de logique

Vous lancez jenga build depuis un sous-dossier de votre dépôt et obtenez « projet inconnu », alors que la même commande fonctionne depuis la racine.

1. Énumérez deux explications — l'une sur le fichier chargé, l'autre sur ce qu'il contient.
2. Quelle commande, et quelle ligne de sa sortie, les distingue en une seconde ?
3. Une troisième explication n'a rien à voir avec Jenga : laquelle, et qu'est-ce qui la trahit ?

La construction : ordre, fraîcheur, parallélisme

C'est le chapitre le plus important du livre, et sa dernière section est une démonstration qu'il faut refaire soi-même au moins une fois.

9.1 L'ordre

Ce qu'il faut retenir

Jenga trie les projets par leurs dépendances — un *tri topologique* — et affiche le résultat :

Capture réelle

```
1 Build Order (1 projects):
2   1. Bonjour [CONSOLE_APP]
```

Cette ligne est un contrat. Ce qui n'y figure pas ne sera pas construit, et aucune modification de ses sources n'aura d'effet.

Le piège

« **Le build est vert** » ne dit rien de ce qui a été construit. Mesuré sur ce dépôt : un développeur modifie un module, lance une construction, obtient `Projects Built: 19/19` et `SUCCESS` — et le fichier objet de sa modification **n'existait pas**. La cible construite ne tirait pas son module.

Un build vert prouve que ce qu'il a construit compile, pas que ce que vous avez écrit a été construit.

Trois vérifications, et elles sont bon marché : le `.obj` existe et est plus récent que la source ; une chaîne caractéristique de votre modification est dans la bibliothèque produite ; la ligne `Compiled: <votre fichier>` apparaît dans la sortie.

9.2 Ce qui est recompilé, et ce qui ne l'est pas

Ce qu'il faut retenir

La décision se prend fichier par fichier, et voici la fonction qui la prend :

Jenga/Core/Builder.py:1380-1405

```
1  if not obj.exists():
2      return True
3
4  obj_mtime = obj.stat().st_mtime
5  if src.stat().st_mtime > obj_mtime:
6      return True
7
8  dep_file = self.GetDependencyFilePath(objectFile)
9  if not dep_file.exists():
10     return True
11
12  deps = self._ParseDependencyFile(dep_file, project)
13  if not deps:
14     return True
15
16  for dep in deps:
17     if not dep.exists():
18         return True
19     if dep.stat().st_mtime > obj_mtime:
20         return True
```

Quatre raisons de recompiler, dans l'ordre où elles sont testées : l'objet n'existe pas ; la source est plus récente que lui ; le fichier de dépendances manque ; un en-tête dont il dépend est plus récent.

Ce qu'il faut retenir

Les en-têtes sont suivis, et c'est le compilateur qui les liste. Le fichier de dépendances est produit par le compilateur lui-même à la compilation précédente : Jenga ne devine pas quels `#include` vous avez écrits, il lit ce que le compilateur a réellement ouvert. C'est pourquoi la *première* construction est plus lente : il n'y a pas encore de fichier de dépendances, donc tout est recompilé.

9.3 La décision se prend sur les DATES

Le piège central de Jenga, et il est mesurable

Regardez à nouveau la fonction ci-dessus : elle compare des `st_mtime` — des **dates**. Pas des empreintes.

Conséquence : **une source modifiée dont la date est ANCIENNE est déclarée à jour**, et l'ancien binaire tourne.

Cela n'a rien de théorique. Une date ancienne arrive dès qu'on restaure un fichier :

Geste	Ce qu'il fait à la date
<code>cp -p, rsync -t</code>	la préserve
extraire une archive tar, zip	la préserve
Copy-Item (PowerShell)	la préserve
<code>git checkout</code>	la met à <i>maintenant</i> — sûr
<code>cp</code> sans <code>-p</code>	<i>maintenant</i> — sûr

La démonstration

Voici l'expérience, faite sur le projet du chapitre 3. À chaque étape on modifie la source, on lance `jenga build`, et on lance le programme :

Capture réelle – ce que le PROGRAMME affiche

```
1 1. source = VERSION UN, construite normalement      -> VERSION UN
2 2. source = VERSION DEUX, date NEUVE                -> VERSION DEUX
3 3. source = VERSION TROIS, date REMISE A L'ANCIENNE -> VERSION DEUX
4 4. la meme source, apres `touch`                   -> VERSION TROIS
```

Ce qu'il faut retenir

Lisez la ligne 3 deux fois. Le fichier sur le disque dit VERSION TROIS. Le programme dit VERSION DEUX. La construction a répondu SUCCESS.

Et la ligne 4 montre le remède : un simple `touch` — qui ne change pas un octet du contenu — suffit à déclencher la recompilation.

Le témoin choisi, et pourquoi celui-là

Le témoin n'est PAS la sortie de `jenga`. Une ligne « déjà à jour » serait déjà une réponse, et l'on ne saurait pas si elle est juste.

Le témoin est ce que le programme affiche : une compilation qui n'a pas eu lieu et une compilation sans effet se ressemblent dans un journal ; elles ne se ressemblent pas à l'écran.

Il existe un moteur à empreintes, et il ne sert pas

Jenga porte un module `Jenga/Core/Incremental.py`, 227 lignes, qui calcule des sha256 — et son propre commentaire dit pourquoi :

Jenga/Core/Incremental.py:36-40

```
1 def ComputeFileHash(filepath: str, algorithm: str = "sha256") -> str:
2     """Calcule le hash d'un fichier. Utilise pour detecter les changements.
3     Par défaut sha256, plus fiable que mtime seul."""
```

Mesuré : aucune de ses méthodes n'est appelée. Le module est importé par `Daemon.py` et par `Core/__init__.py`, et une recherche de `Incremental` hors du fichier lui-même rend zéro. La méthode `Cache.UpdateIncremental` porte un nom voisin mais fait autre chose : elle recharge la *description*, pas les sources.

C'est un doublon dont un seul côté est vivant — et l'on croit lire un système à empreintes en lisant un système à dates. *Une capacité écrite n'est pas une capacité employée ; seul un appelant le prouve.*

Ce qu'il faut en faire

Situation	Le geste
après toute restauration de fichiers	touch les fichiers restaurés
un doute sur la fraîcheur d'un binaire	jenga rebuild
avant une campagne de mesures	rebuild, puis vérifier une date
un correctif « sans effet »	vérifier le .obj avant d'accuser le code

Ce qu'il faut retenir

Le pire de ce piège n'est pas le temps perdu : c'est la fausse conclusion. On retire un correctif juste, parce qu'« il ne change rien ». Le défaut coûte alors deux fois — une fois à ne pas s'appliquer, une fois à faire défaire une bonne chose.

9.4 Le parallélisme

Ce qu'il faut retenir

Les fichiers d'un même projet se compilent en parallèle ; les projets, eux, suivent l'ordre des dépendances. C'est la seule combinaison possible : deux projets indépendants *pourraient* aller de front, mais un projet et sa dépendance, non.

Conséquence pratique : **un graphe très en chaîne se parallélise mal**. Trente projets dont chacun dépend du précédent se construisent en trente étapes, quel que soit le nombre de cœurs.

Le piège

En construction parallèle, l'ordre des messages n'est pas l'ordre des événements. Une erreur affichée après un succès peut le précéder dans le temps.

Et par défaut, **la construction s'arrête au premier échec** — il y a donc exactement un échec par relevé. C'est utile à savoir avant de compter les erreurs : `Failed: 1` ne veut pas dire qu'un seul fichier est cassé, mais qu'on s'est arrêté au premier.

--keep-going continue et donne le compte réel.

9.5 Nettoyer

jenga clean	supprime les objets et les artefacts
jenga rebuild	clean puis build

Ce qu'il faut retenir

rebuild est la réponse à toute question de fraîcheur. Il coûte du temps et il tranche : après lui, ce qui tourne vient de ce qui est sur le disque.

C'est la commande à lancer avant de conclure qu'un correctif ne marche pas — et avant toute mesure de performance qu'on publiera.

Ce chapitre en bref

Jenga trie les projets par dépendances et affiche l'ordre : ce qui n'y figure pas ne sera pas construit, et un build vert ne prouve pas que *votre* fichier l'a été. La recompilation se décide sur quatre critères, tous fondés sur des **dates** — pas sur des empreintes, bien qu'un moteur à sha256 existe dans le code, dormant. Une source restaurée avec une date ancienne est donc déclarée à jour : c'est démontré, et le remède est `touch` ou `rebuild`. Enfin les fichiers se parallélisent, les projets non, et la construction s'arrête au premier échec sauf `--keep-going`.

Questions de cours

1. Quelles sont les quatre raisons de recompiler un fichier ?
2. Sur quoi la décision se fonde-t-elle : dates ou empreintes ?
3. Citez trois gestes qui préservent la date d'un fichier.
4. Pourquoi `Failed: 1` ne signifie-t-il pas « un seul fichier cassé » ?
5. Que prouve un build vert, et que ne prouve-t-il pas ?

Exercice pratique

Reproduisez la démonstration des dates, vous-même, sur votre projet.

1. construisez, lancez, notez la sortie ;
2. modifiez la source normalement, reconstruisez, lancez : la sortie change ;
3. modifiez-la à nouveau *en remettant l'ancienne date* (`os.utime` en Python, `touch -r` en shell), reconstruisez, lancez ;
4. `touch`, reconstruisez, lancez.

Notez les quatre sorties du programme, pas celles de Jenga.

Exercice de démonstration

Prouvez que le module à empreintes ne sert pas.

1. Cherchez dans le dépôt de Jenga les appels à `Incremental`. en excluant le fichier lui-même. Notez le compte.
2. Cherchez qui *importe* le module. Notez que `l'import` existe.
3. Concluez sur la différence entre les deux comptes.



Le contrôle positif est obligatoire avant de croire un zéro : lancez d'abord la même recherche sur un nom dont vous savez qu'il est employé — `Builder`. par exemple. Si elle rend zéro aussi, c'est votre commande qui ne sait pas chercher, et le premier zéro ne prouvait rien.

Exercice de logique

Vous corrigez un défaut, reconstruisez, et le comportement ne change pas.

1. Énumérez au moins cinq causes, en séparant celles où la compilation *n'a pas eu lieu*

de celles où elle a eu lieu *sans effet*.

2. Rangez-les par **coût de vérification**, du moins cher au plus cher — et non par probabilité.
3. Justifiez ce critère : pourquoi, quand on ne sait rien, vaut-il mieux ordonner par coût que par probabilité ?

Ce que cette partie vous laisse

Un .jenga n'est pas analysé : il est exécuté, et le graphe existe quand l'exécution se termine. La construction trie ensuite les projets par leurs dépendances, parallélise les fichiers d'un même projet, et décide de recompiler sur des **dates**. Retenez les deux phrases qui vous éviteront le plus d'heures : *un build vert prouve que ce qu'il a construit compile, pas que ce que vous avez écrit a été construit* ; et *après toute restauration de fichiers, touch*.

PARTIE IV

Les cibles

Douze systèmes, et trois états

Jenga déclare douze systèmes cibles. Cette partie dit ce que chacun demande — et surtout où **il en est**, car les douze ne sont pas au même stade.

Trois mots servent tout au long, et il faut les trois :

déclarée des lignes existent; rien ne dit qu'elles fonctionnent

construite un artefact sort; rien ne dit qu'il se lance

éprouvée l'artefact a tourné sur la cible

Annoncer «douze plateformes» sans cette distinction est une promesse qu'on ne peut pas tenir. Le chapitre 12 donne l'état réel de chacune, et l'exemple mesuré d'une plateforme qu'un descripteur annonçait sans avoir le moindre filtre pour elle.

Le bureau : Windows, Linux, macOS

Trois systèmes, trois façons de compiler, et un descripteur unique. Ce chapitre donne ce que chacun exige — et ce qu'il exige *de la machine*, qui n'est pas la même chose.

10.1 Les chaînes d'outils disponibles

jenga info – capture réelle, extrait

1	Name	Family	Target OS	Arch	Env
2	=====				
3	host-clang	clang	Windows	x86_64	mingw
4	host-gcc	gcc	Windows	x86_64	mingw
5	msvc	msvc	Windows	x86_64	msvc
6	clang-mingw	clang	Windows	x86_64	mingw
7	mingw	gcc	Windows	x86_64	mingw
8	clang-cross-linux	clang	Linux	x86_64	gnu
9	android-ndk	android-ndk	Android	arm64	android
10	emscripten	emscripten	Web	wasm32	

Ce qu'il faut retenir

Cette table est celle de LA MACHINE, pas celle de Jenga. Android et Emscripten y figurent parce que leurs SDK sont installés ici. Sur une machine neuve, la table est plus courte — et un descripteur qui exige une chaîne absente échoue au chargement. Jenga en déclare onze au total; la table montre celles qu'il a *trouvées*.

10.2 Windows

Applications/NkDames/NkDames.jenga:113-129 – cite tel quel

```

1  with filter("system:Windows && !options:windows-runtime=uwp && !system:XboxSeries &&
    !system:XboxOne"):
2      windowedapp()
3      usetoolchain(TC_WINDOWS)
4      defines(["WIN32_LEAN_AND_MEAN", "_UNICODE", "UNICODE", _VK_ON])
5      if _VULKAN_LIB:
6          libdirs([_VULKAN_LIB])
7      _WIN_LINKS = [
8          "user32", "gdi32", "opengl32", "dwmapi", "shell32",
9          "mf", "mfplat", "mfreadwrite", "mfuuid",
10         "uuid", "ole32", "dinput8", "dxguid",
11         "d3d11", "d3d12", "dxgi", "d3dcompiler",
12         "avrt", "winmm", # NKAudio (WASAPI / MMCSS)
13     ]
14     if _WANT_VULKAN:
15         _WIN_LINKS.append("vulkan-1")
16     links(_WIN_LINKS)

```

Ce bloc est la thèse du livre en huit lignes

Rien ici n'est une « fonctionnalité de Jenga ». La liste est construite par du Python ordinaire : on l'écrit, on lui ajoute conditionnellement une entrée, puis on la *pass*e à `links()`. C'est la conséquence directe du chapitre 4 — un `.jenga` est exécuté. Un système de construction à syntaxe déclarative demanderait ici une notion spéciale de « liste conditionnelle ». Il n'y en a pas besoin : `if` et `append` existaient déjà.

TC_WINDOWS n'est pas de Jenga



Ne le tapez pas tel quel : vous obtiendrez un `NameError`. `usetoolchain` prend une chaîne — `def usetoolchain(name: str), Jenga/Core/Api.py:1836` — et les exemples livrés avec Jenga écrivent `usetoolchain("clang-mingw")`. `TC_WINDOWS` est une variable de notre dépôt :

config/toolchain.jenga:14

```
1 TC_WINDOWS = "nk-windows-clang-mingw"
```

Un `.jenga` étant du Python, une variable définie dans un fichier inclus est visible partout ensuite — c'est ce qui permet de nommer une fois la chaîne d'outils et de la changer pour **189 descripteurs** d'un coup. Le prix est qu'à la lecture, rien ne distingue un symbole de Jenga d'un symbole maison.

C'est le deuxième de la série, après `nkentseudependson` (chapitre 7). Devant un nom que la documentation de Jenga ignore, la question est toujours la même : *où est-il défini* ? Un `grep` sur `<nom>` = dans vos propres `.jenga` répond en une seconde.

Ce qu'il faut retenir

WIN32_LEAN_AND_MEAN n'est pas de la superstition. Sans lui, `windows.h` tire des dizaines de milliers de lignes dont vous n'avez pas besoin — et surtout des **macros** : `min`, `max`, `near`, `far`, `small`. Chacune casse silencieusement un code qui emploie le même nom. **_UNICODE** et **UNICODE** choisissent les variantes larges des API. Les deux, parce que l'un est lu par la bibliothèque C et l'autre par l'API Windows — en déclarer un seul donne un mélange qui compile et se comporte mal sur les chemins accentués.

Les bibliothèques se déclarent par leur nom nu

`"user32"` et non `"user32.lib"`. Jenga ajoute ce qu'il faut selon la chaîne : `-luser32` pour MinGW, `user32.lib` pour MSVC. C'est ce qui rend la ligne identique pour les deux.

10.3 Linux, et ses quatre backends

C'est le cas le plus instructif du livre, parce qu'un même système y demande quatre descriptions.

Applications/NkDames/NkDames.jenga – reel, abregé

```
1 with filter("system:Linux && options:linux-backend=xlib || "  
2             "system:Linux && !options:linux-backend && !options:headless"):  
3     usetoolchain("clang-native")
```

```

4 objdir("../Build/Obj/{cfg.buildcfg}-{cfg.system}-xlib/{prj.name}")
5 targetdir("../Build/Bin/{cfg.buildcfg}-{cfg.system}-xlib/{prj.name}")
6 defines(["NKENTSEU_FORCE_WINDOWING_XLIB_ONLY"])
7 links(["pthread", "X11", "Xext", "Xrandr", "GL", "asound"])

```

Backend	Ce qu'il lie	Sortie
xlib	X11 Xext Xrandr GL asound	Release-Linux-xlib
xcb	xcb xcb-image xcbcommon...	Release-Linux-xcb
wayland	wayland-client wayland-egl...	Release-Linux-wayland
headless	pthread seulement	Release-Linux-headless

Ce qu'il faut retenir

Chaque backend a son propre dossier de sortie, et c'est indispensable : sans le suffixe, construire pour Wayland écraserait la construction XLib, et l'on lancerait un binaire lié aux mauvaises bibliothèques.



Le corollaire coûte cher aux scripts : le dossier n'est pas Release-Linux mais Release-Linux-xlib. Un empaceteur qui code le premier en dur échoue sur une construction réussie — mesuré sur ce dépôt.

Le second membre de l'expression est le défaut

Relisez la condition : « backend xlib demandé, *ou* aucun backend demandé et pas de mode sans écran ». Le second membre attrape l'utilisateur qui n'a rien choisi. C'est la façon d'écrire un défaut avec des filtres — il n'y a pas de mot-clé « sinon ».

Le piège

Construire pour Linux depuis Windows ne marche pas avec cette description : clang-native y cherche X11/X.h, qui n'existe pas. Deux issues, et le dépôt emploie les deux : **WSL2**, où les en-têtes existent, pour les constructions locales; une **intégration continue** Linux pour le reste. Aucune n'est un contournement : ce sont des machines Linux.

10.4 macOS

Applications/NkDames/NkDames.jenga – reel

```

1 with filter("system:macOS"):
2     windowedapp()
3     usetoolchain("clang-native")
4     frameworks(["Cocoa", "QuartzCore", "OpenGL", "Metal",
5                 "AudioToolbox", "CoreAudio", "AudioUnit"])

```

Ce qu'il faut retenir

Pas de links, des frameworks. Un framework Apple porte à la fois les en-têtes et le binaire. Le commentaire du descripteur note que cette liste a été établie *sur l'édition de liens de l'intégration continue*, pas devinée — et il précise le point qui surprend : « les 23 projets

COMPILENT sans ces frameworks; seul le lien final tombe». *Une dépendance de framework ne se voit pas à la compilation. C'est pourquoi elle s'oublie, et pourquoi elle ne se découvre qu'à la toute fin.*

Le piège

Construire pour macOS exige macOS — et Xcode. Jenga déclare des chaînes de compilation croisée, mais le SDK Apple ne se redistribue pas. Le dépôt passe donc par une intégration continue macOS, et son script d'empaquetage le dit en toutes lettres plutôt que de laisser croire le contraire.

10.5 Debug et Release

La forme, identique dans tous les descripteurs

```
1 with filter("config:Debug"):
2     defines(["_DEBUG", "DEBUG"]); optimize("Off"); symbols(True)
3 with filter("config:Release"):
4     defines(["NDEBUG"]); optimize("Speed"); symbols(False)
```

Ce qu'il faut retenir

Debug et Release ne sont pas le même programme, et il faut le prendre au sérieux : les assert disparaissent, la mémoire non initialisée change de contenu, et l'optimiseur expose des comportements indéfinis que le Debug masquait.

Un défaut qui n'existe que dans l'une des deux configurations est fréquent. Toute mesure, tout chiffre, toute campagne doit donc porter sa configuration — sans quoi deux mesures justes se contredisent sans que personne ait tort.

Ce chapitre en bref

Windows demande WIN32_LEAN_AND_MEAN et les deux macros Unicode, et ses bibliothèques se nomment nues. Linux n'est pas une cible mais quatre, chacune avec son dossier de sortie suffixé — ce que les scripts doivent lire et non coder en dur — et le second membre du filtre y sert de valeur par défaut. macOS emploie des frameworks, dont la liste ne se découvre qu'à l'édition de liens. Enfin Debug et Release produisent deux programmes différents : une mesure sans sa configuration n'est pas une mesure.

Questions de cours

1. La table des chaînes d'outils décrit-elle Jenga ou la machine ?
2. Que se passe-t-il sans WIN32_LEAN_AND_MEAN ?
3. Pourquoi chaque backend Linux a-t-il son propre dossier de sortie ?
4. Pourquoi un framework macOS manquant ne se voit-il qu'à la fin ?
5. Citez trois différences réelles entre Debug et Release.

Exercice pratique

Faites construire votre projet du chapitre 3 sur deux configurations et vérifiez, *sur le disque*, que les deux artefacts coexistent.

Puis ajoutez un filtre Linux avec une option `--mon-backend` à trois valeurs, chacune posant une définition différente et un dossier de sortie différent.

Vérifiez par `jenga compile-flags` — pas par la construction, que vous ne pouvez peut-être pas faire ici. **C'est justement l'intérêt** : on peut vérifier une description pour une plateforme qu'on n'a pas.

Exercice de démonstration

Prouvez que Debug et Release ne produisent pas le même programme.

Écrivez un programme dont le comportement dépend de `NDEBUG` — un `assert` qui échoue, par exemple, ou un `#ifdef` qui change un message. Construisez et lancez dans les deux configurations.

Ce que la démonstration doit établir : ce ne sont pas deux vitesses du même programme, ce sont deux programmes.

Puis, plus utile encore : **mesurez la taille des deux exécutables**. L'écart vous donnera une idée de ce que les symboles de débogage coûtent — et vous saurez, la prochaine fois qu'un binaire vous paraît trop gros, quelle question poser en premier.

Exercice de logique

Un collègue rapporte : « le programme plante en Release, mais pas en Debug ».

1. Énumérez trois familles de causes possibles, propres à cette asymétrie.
2. Pourquoi le réflexe « je vais déboguer ça en Debug » est-il précisément celui qui ne trouvera rien ?
3. Proposez deux approches qui gardent le Release — et dites ce que chacune coûte.

Le mobile : Android, iOS, HarmonyOS

Le mobile ajoute quelque chose que le bureau n'a pas : un **manifeste**. Il ne suffit plus de compiler et de lier — il faut déclarer une identité, des permissions, une orientation, et signer. Ce chapitre couvre les trois systèmes.

11.1 Android

Applications/NkDames/NkDames.jenga – reel, abregé

```

1 with filter("system:Android"):
2     windowedapp()
3     usetoolchain("android-ndk")
4     androidapplicationid("com.nkentseu.nkdames")
5     androidisgame(True)
6     androidminsdk(24)
7     androidtargetsdk(34)
8     androidcompilesdk(34)
9     androidabis(["armeabi-v7a", "arm64-v8a", "x86", "x86_64"])
10    androidnativeactivity(True)
11    androidscreenorientation("portrait")
12    androidallowrotation(False)
13    androidstl("c++_shared")
14    links(["android", "log", "EGL", "GLLESv3", "dl", "OpenSLES"])

```

Fonction	Ce qu'elle décide
<code>androidapplicationid</code>	l'identité du paquet — unique sur l'appareil
<code>androidminsdk</code>	la version d'Android la plus ancienne acceptée
<code>androidabis</code>	les architectures embarquées
<code>androidnativeactivity</code>	pas de code Java du tout
<code>androidscreenorientation</code>	ce que le manifeste impose
<code>androidstl</code>	quelle bibliothèque standard C++ embarquer

Ce qu'il faut retenir

androidabis multiplie la taille du paquet. Quatre architectures, c'est quatre bibliothèques natives dans l'APK. Le paquet « universel » d'un jeu de ce dépôt pèse 7 Mo pour 2 Mo sur le bureau.

On les garde pour un APK de test qui s'installe partout ; pour une distribution, on découpe.

Une NativeActivity n'a pas de classes.dex, et c'est normal

`androidnativeactivity(True)` produit un manifeste avec `android:hasCode="false"` : il n'y a **aucun** code Java, donc rien à convertir en dex.



Un contrôle qui exige un `classes.dex` déclarera donc cassé un APK qui s'installe et fonctionne. *Vérifié : les APK de ce dépôt n'en ont pas, et tournent sur un téléphone réel.*

Sans orientation déclarée, l'image sort tournée

Le système impose la sienne. Le descripteur commente d'ailleurs son choix : « PORTRAIT : c'est l'orientation du jeu [...]; la mise en page SAIT faire les deux; c'est un choix de produit, pas une limite technique ».

Et l'on vérifie ce que le paquet porte *réellement*, pas ce que le descripteur demande :

`aapt dump xmltree – capture réelle`

```
1 A: android:screenOrientation(0x0101001e)=(type 0x10)0x1
```

0x1 vaut `SCREEN_ORIENTATION_PORTRAIT`.



Et cherchez la valeur, pas le mot. Dans un manifeste binaire, `screenOrientation` est un **entier** : chercher la chaîne « portrait » rend ABSENT sur un paquet parfaitement verrouillé. Mesuré — et j'ai failli annoncer un défaut sur cet instrument aveugle.

Ce qu'il faut sur la machine

<code>ANDROID_SDK_ROOT</code>	le SDK
<code>ANDROID_NDK_ROOT</code>	le NDK
<code>JAVA_HOME</code>	un JDK 17 — pour empaqueter et signer
<code>~/ .android/debug.keystore</code>	la clé de test

Les deux dernières lignes sont celles qu'on oublie, et leur absence ne casse pas la compilation : elle produit un APK *sans signature* que `adb install` refuse en silence.

Vérification rapide d'un APK : il doit contenir `META-INF/MANIFEST.MF` et un `META-INF/*.RSA`.

11.2 iOS

Applications/NkDames/NkDames.jenga – reel

```
1 with filter("system:iOS"):
2     windowedapp()
3     frameworks(["UIKit", "QuartzCore", "OpenGL", "AVFoundation", "Metal",
4                 "AudioToolbox", "CoreAudio"])
5     iosbundleid("com.nkentseu.nkdames")
6     iosversion("1.0")
7     iosminsdk("14.0")
8     iosbuildnumber(1)
```

La liste des frameworks n'est PAS celle de macOS

AudioUnit n'existe pas sur iOS. C'est un framework distinct sur macOS seulement; sur iOS, ses symboles — `AudioComponent*`, `AudioOutputUnit*` — vivent dans `AudioToolbox`.

Recopier la liste macOS donne ld: framework AudioUnit not found. Mesuré sur l'intégration continue iOS.

Un précédent se vérifie sur SA plateforme. C'est vrai des frameworks Apple, et c'est vrai de bien d'autres choses.

Ce que le descripteur avoue lui-même

Applications/NkDames/NkDames.jenga

 1 # Non verifie sur machine : la construction iOS exige macOS + Xcode.
2 # Ces lignes suivent le modele de NKPA, qui est dans le meme cas. Elles
3 # rendent la cible DECLAREE, pas EPROUVEE.

C'est la distinction que ce livre tient partout. Une cible déclarée a des lignes dans un descripteur; une cible éprouvée a produit un artefact qu'on a lancé. Les confondre fait annoncer un support qu'on n'a pas.

Le même commentaire note que le filtre iOS **n'existait pas** jusqu'au 2026-08-27, alors que l'en-tête du fichier annonçait « Mobile : Android, iOS, HarmonyOS » — *une plateforme qu'on croit couverte parce qu'un commentaire la nomme.*

11.3 HarmonyOS

Applications/NkDames/NkDames.jenga – reel

```
1 with filter("system:HarmonyOS"):
2     windowedapp()
3     usetoolchain("ohos-ndk")
4     harmonybundlename("com.nkentseu.nkdames")
5     harmonyminsdk(12)
6     harmonytargetapi(12)
7     harmonyorientation("portrait")
8     links(["hilog_ndk.z", "EGL", "GLSv3", "vulkan"])
```

Ce qu'il faut retenir

La structure est celle d'Android, les noms changent : harmonybundlename pour l'identité, hilog_ndk.z pour le journal — l'équivalent de log.

Et l'orientation s'y déclare pour la même raison : sans elle, le système impose la sienne et l'image sort tournée d'un quart de tour.

Le piège

jenga package --platform harmonyos exige un hvigor-config.json5 que les applications de ce dépôt n'ont pas. Leur .hap vient donc de jenga build, pas de package. C'est écrit dans le script d'emballage, et c'est le genre de chose qu'il vaut mieux découvrir dans un livre que dans un journal d'erreurs.

11.4 Le point commun des trois

Ce qu'il faut retenir

Sur mobile, la construction n'est que la moitié du travail. Il reste à produire un manifeste juste, à embarquer les bonnes architectures, et à signer.

Et ces trois-là échouent différemment de la compilation : **ils ne produisent pas d'erreur de compilateur.** L'APK se construit, il ne s'installe pas ; le manifeste se génère, il déclare la mauvaise orientation ; le paquet existe, il n'est pas signé.

La vérification d'un artefact mobile porte donc sur le PAQUET, jamais sur la sortie de la construction.

Ce chapitre en bref

Le mobile ajoute un manifeste : identité, permissions, orientation, signature. Android se décrit par une quinzaine d'appels dont l'oubli le plus coûteux est l'orientation ; ses APK NativeActivity n'ont légitimement pas de `classes.dex`, et il faut vérifier la valeur dans le manifeste binaire, pas le mot. iOS partage la forme de macOS mais pas sa liste de frameworks — AudioUnit n'y existe pas — et le descripteur avoue être déclaré sans être éprouvé. HarmonyOS reprend la structure d'Android sous d'autres noms. Retenez surtout que ces trois systèmes échouent *après* la compilation, donc que la vérification porte sur le paquet.

Questions de cours

1. Que produit `androidnativeactivity(True)` dans le manifeste, et quelle conséquence sur le contenu de l'APK ?
2. Comment vérifier l'orientation réellement déclarée par un APK ?
3. Pourquoi AudioUnit casse-t-il la construction iOS ?
4. Quelle différence entre une cible *déclarée* et une cible *éprouvée* ?
5. Qu'est-ce qui manque à un APK quand `JAVA_HOME` n'est pas défini ?

Exercice pratique

Prenez un APK — le vôtre, ou l'un de ceux du dépôt — et établissez quatre faits par la commande, pas par le descripteur :

1. son identité de paquet ;
2. son orientation déclarée ;
3. les architectures qu'il embarque ;
4. s'il est signé.

`aapt dump badging` et `aapt dump xmltree` répondent aux trois premières ; la quatrième se lit dans la liste des fichiers du zip.

Exercice de démonstration

Prouvez qu'un manifeste binaire ne se lit pas comme du texte.

1. Cherchez la chaîne `portrait` dans `AndroidManifest.xml` extrait d'un APK verrouillé en

portrait. Notez le résultat.

2. Faites lire le même fichier par `aapt dump xmltree`. Notez la valeur de `screenOrientation`.

Ce que la démonstration établit : la première méthode dit ABSENT sur un paquet parfaitement correct. Une valeur d'énumération y est un *entier*.



Et posez le contrôle positif : cherchez une chaîne dont vous savez qu'elle est là — `NativeActivity`, par exemple. Si elle non plus ne sort pas, votre lecteur est aveugle et le premier résultat ne prouvait rien.

Exercice de logique

Votre APK se construit, mais `adb install` le refuse sans message clair.

1. Énumérez quatre causes possibles, en distinguant celles qui viennent de la construction de celles qui viennent de l'appareil.
2. Pour chacune, dites ce que vous regarderiez *dans le paquet* — et non dans la sortie de la construction.
3. Une cause ne se voit ni dans le paquet ni dans la construction. Laquelle, et où la prendriez-vous ?

Le Web, et ce qui est déclaré sans être éprouvé

Deux sujets dans ce chapitre, et ils vont ensemble : une cible qui fonctionne et qu'on connaît mal, et six cibles dont il faut dire honnêtement ce qu'on en sait.

12.1 Le Web

Applications/NkDames/NkDames.jenga – reel

```
1 with filter("system:Web"):
2     consoleapp()
3     usetoolchain("emscripten")
4     defines(["NKENTSEU_ENABLE_EMSCRIPTEN"])
5     emscriptencanvasid("canvas")
6     emscripteninitialmemory(32)
7     emscriptenextraflags([
8         "-s", "ASYNCIFY",
9         "-s", "ALLOW_MEMORY_GROWTH=1",
10        "-s", "FULL_ES3=1",
11        "-s", "USE_WEBGL2=1",
12        "-s", "NO_EXIT_RUNTIME=1",
13    ])
```

Ce qu'il faut retenir

consoleapp() sur le Web, alors que le bureau dit **windowedapp()**. Ce n'est pas une erreur : la notion de fenêtre native n'existe pas dans un navigateur, la page est la fenêtre. C'est un bon exemple de ce que les filtres servent à faire — redéfinir même le genre du projet selon la cible.

Drapeau	Ce qu'il change
ASYNCIFY	permet à la boucle de rendre la main
ALLOW_MEMORY_GROWTH	la mémoire n'est plus figée au démarrage
FULL_ES3 + USE_WEBGL2	la traduction OpenGL ES vers WebGL2
NO_EXIT_RUNTIME	le programme ne se termine pas à la fin du main

ASYNCIFY n'est pas une option de confort

Un navigateur exécute votre page sur *son* fil principal — celui qui dessine et qui répond aux clics. Une boucle infinie n'y rend jamais la main : l'onglet gèle, et le navigateur finit par proposer de le tuer.

ASYNCIFY rend possible de suspendre et reprendre le programme, ce qui permet à la boucle de céder. **Sans lui, votre programme est correct et votre onglet est mort.**



Et le drapeau ne suffit pas : encore faut-il que la boucle *cède réellement*. Un mécanisme de temporisation qui existe et qu'on n'appelle pas laisse le même symptôme.

Le piège

Un `.wasm` ouvert en double-cliquant le `.html` ne se charge pas. Les navigateurs refusent le protocole `file://` pour ce genre de ressource.

Il faut *servir* le dossier par HTTP — `python -m http.server` suffit. Le message d'erreur, lui, parle de *CORS* ou de *fetch*, et n'aide pas à trouver la cause.

Ce qu'il faut retenir

`emscripteninitialmemory(32)` donne 32 Mo au départ. Avec `ALLOW_MEMORY_GROWTH`, ce n'est qu'un point de départ — mais un point de départ trop petit fait payer plusieurs ré-allocations pendant le chargement, qui est justement le moment où l'utilisateur regarde.

12.2 Ce qui est déclaré, et ce qui est éprouvé

C'est la section la plus honnête du livre, et la plus utile avant une promesse.

Cible	État	Ce qui le fonde
Windows	éprouvée	artefacts construits et lancés ici
Linux	éprouvée	construits par WSL2, binaires lancés
Web	éprouvée	<code>.wasm</code> produit, page servie
Android	éprouvée	APK installé sur un téléphone réel
HarmonyOS	construite	<code>.hap</code> produit, non installé ici
macOS	déclarée	exige macOS + Xcode — CI
iOS	déclarée	idem, et le descripteur le dit
PS4, PS5, Xbox, Switch	déclarées	SDK sous accord, non testés ici
FreeBSD, OpenBSD	déclarées	aucune mesure

Ce qu'il faut retenir

Trois états, et il faut les trois mots.

Déclarée : des lignes existent dans un descripteur et une chaîne d'outils est prévue. Rien ne dit qu'elles fonctionnent.

Construite : un artefact sort. Rien ne dit qu'il se lance.

Éprouvée : l'artefact a tourné sur la cible.

Annoncer « douze plateformes » sans cette distinction est une promesse qu'on ne peut pas tenir. Et l'écart n'est pas théorique : le filtre iOS de ce dépôt *n'existait pas* alors que l'en-tête du fichier annonçait iOS depuis des mois.

Le piège

Un commentaire qui nomme une plateforme n'est pas un support. Le cas est mesuré : l'en-tête disait « Mobile : Android, iOS, HarmonyOS », et aucun filtre iOS n'existait dans le fichier.

Rien ne le signalait : **la construction bureau ne dit rien d'un filtre manquant**. On croit couvrir une plateforme parce qu'une phrase la nomme.
Le contrôle est immédiat : `grep "system:iOS"` dans le descripteur. S'il n'y a pas de filtre, il n'y a pas de cible.

12.3 Les consoles

Ce qu'il faut retenir

Jenga déclare PS4, PS5, Xbox One, Xbox Series et Switch, avec leurs fonctions : `gdkpath`, `xboxmode`, `xboxpackagename`, `xboxsigningmode`...

Leurs SDK ne sont ni publics ni redistribuables. Le code qui les gère existe dans `Jenga/Core/Builders/`, il n'a pas pu être exercé ici, et ce livre ne prétendra pas le contraire.

Ce que vous pouvez en faire aujourd'hui : lire les descripteurs comme un modèle de ce qu'une cible sous accord demande — une identité de paquet, un mode de signature, un chemin de SDK.

12.4 Choisir sa cible en ligne de commande

```
1 jenga build --platform windows
2 jenga build --platform linux --linux-backend=wayland
3 jenga build --platform android --config Release
4 jenga build --platform web
```

Ce qu'il faut retenir

Sans `--platform`, Jenga construit pour la machine courante. C'est le défaut le plus sûr, et c'est aussi pourquoi on oublie qu'il existe : on peut croire tester la portabilité en ne construisant jamais que pour soi.

Ce chapitre en bref

Le Web se décrit comme une console — la page est la fenêtre — et exige `ASYNCIFY`, sans quoi la boucle gèle l'onglet; son artefact doit être servi par HTTP, jamais ouvert en `file://`. Les douze cibles déclarées ne sont pas au même stade : *déclarée*, *construite* et *éprouvée* sont trois états distincts, et les confondre est une promesse qu'on ne tient pas. Un commentaire qui nomme une plateforme n'est pas un support — mesuré sur ce dépôt, où iOS était annoncé sans qu'aucun filtre n'existe.

Questions de cours

1. Pourquoi le Web est-il déclaré `consoleapp()` ?
2. Que se passe-t-il sans `ASYNCIFY` ?
3. Pourquoi un `.wasm` ne se charge-t-il pas en `file://` ?
4. Distinguez « déclarée », « construite » et « éprouvée ».
5. Comment vérifier en une commande qu'une plateforme est réellement décrite ?

Exercice pratique

Construisez votre projet pour le Web, puis servez-le et ouvrez-le.

1. `jenga build --platform web`;
2. trouvez les fichiers produits — combien, et lesquels?;
3. ouvrez le `.html` en double-cliquant, et notez le message;
4. servez le dossier par HTTP, ouvrez, et notez la différence.

Le point 3 est le plus formateur : **gardez le message**, vous le reverrez.

Exercice de démonstration

Prouvez qu'un commentaire n'est pas un support.

Prenez un descripteur du dépôt et dressez deux listes :

1. les plateformes que son **en-tête** annonce;
2. les plateformes pour lesquelles un `with filter("system:...")` existe réellement.

Ce que la démonstration établit : les deux listes peuvent différer, et rien dans la construction ne le signale.

Refaites l'exercice sur *votre* projet dès qu'il annonce une plateforme. C'est un contrôle d'une minute qui empêche une promesse fausse.

Exercice de logique

On vous demande : « le projet supporte-t-il macOS ? »

1. Quelles trois réponses différentes pourriez-vous donner, selon l'état réel ?
2. Pour chacune, quelle preuve produiriez-vous ?
3. L'une de ces preuves ne peut pas être produite sur votre machine. Laquelle, et que répondez-vous alors — sans mentir ni renoncer ?

Ce que cette partie vous laisse

Le bureau demande peu et exige beaucoup de la machine : Linux n'est pas une cible mais quatre, chacune avec son dossier de sortie suffixé. Le mobile ajoute un manifeste, et ses échecs arrivent *après* la compilation — donc la vérification porte sur le paquet, pas sur la construction. Le Web exige ASYNCIFY et un serveur HTTP. Et sur les douze cibles, quatre sont éprouvées ici, une est construite, sept sont déclarées : le dire est plus utile que de les compter.

PARTIE V

Au-delà de la construction

Vingt-sept commandes, pas une

`build` est la commande qu'on connaît. Les vingt-six autres font le reste du métier : lancer, tester, déboguer, surveiller, emballer, signer, déployer, inspecter, générer.

Cette partie les couvre par usage plutôt que par ordre alphabétique — ce qu'on fait après avoir construit, ce qu'on fait pour livrer, ce qu'on fait quand on veut savoir.

Un fil les traverse : plusieurs de ces commandes réussissent en ne faisant rien, ou en faisant autre chose que ce qu'on croit. `test` sans tests, `package` qui écrase un artefact, `gen` qui produit un fichier qui ne suivra plus. Chaque chapitre nomme le témoin qui distingue « ça a marché » de « ça a fait ce que je voulais ».

Lancer, tester, déboguer, surveiller

Construire n'est qu'une des vingt-sept commandes. Ce chapitre couvre les quatre qui servent tous les jours après.

13.1 run — lancer sans connaître le chemin

```
1 jenga run --config Release
2 jenga run --project MonJeu --config Debug
3 jenga run -- --mode=solo --verbose
```

Ce qu'il faut retenir

Le double tiret sépare vos arguments de ceux de Jenga. Tout ce qui suit -- est passé au programme.

Sans lui, `jenga run --verbose` rend Jenga bavard et ne transmet rien. C'est une convention courante, et elle se rappelle par ce symptôme : « mon programme ignore ses options ».

Capture réelle

```
1 EXECUTION -- Bonjour.exe
2 ...\\Build\\Bin\\Release-Windows\\Bonjour\\Bonjour.exe
3
4 Application console sans terminal : ouverture d'une console dediee.
```

Ce qu'il faut retenir

Cette dernière ligne explique une confusion fréquente. Une application de type console lancée sans terminal attaché obtient une console à elle — et sa sortie n'apparaît donc pas dans *votre* fenêtre.

Si vous capturez la sortie d'un programme par un script, vous ne verrez rien. *Ce n'est pas que le programme est muet; c'est qu'il parle ailleurs.*

13.2 test — et ce qu'il répond quand il n'y a rien

jenga test – capture réelle, code de sortie 1

```
1 No test projects found.
```

Ce qu'il faut retenir

Code de sortie 1, message explicite. C'est le bon comportement : ne rien trouver n'est pas un succès. Une intégration continue qui lance jenga test sur un dépôt sans tests doit rougir, pas passer.

« Zéro test exécuté » et « zéro test en échec » se ressemblent au premier coup d'œil et n'ont rien à voir.

Declarer une suite de tests

```
1 with project("MesTests"):
2     testsuite()
3     language("C++")
4     files(["tests/**/*.cpp"])
5     dependson(["MaBibliotheque"])
```

Le cadre de test est fourni par Jenga

Jenga embarque **Unittest** — [Jenga/Unittest/](#) — et le workspace peut l'activer :

```
1 with unittest() as u:
2     u.Precompiled()
```



C'est une confusion mesurée sur ce dépôt : une recherche du cadre de test dans les sources de Nkentseu avait conclu « ce cadre n'existe pas ». Il existe — *dans Jenga*. La recherche était exhaustive et sa racine était fausse.

Une recherche exhaustive ne vaut que ce que vaut sa racine.

Ce qu'il faut retenir

Deux réglages coupent les tests, et il faut savoir qu'ils existent avant de conclure qu'une suite « ne tourne pas » :

<code>disableunittestcompilation() / dutc()</code>	ne les compile plus
<code>disableunittestexecution() / dute()</code>	les compile, ne les lance pas

Le second est le plus trompeur : tout se construit, rien ne s'exécute. Depuis la version 2.4.0, test et run acceptent `--force` pour lever ces deux verrous.

Le piège

Un enchaînement A || B attribue la sortie de B à A. Mesuré : un développeur lance `jenga test --project X || jenga build --target X`, lit 6/6 --- SUCCESS, et conclut que `jenga test` annonce un succès sur une suite qu'il n'exécute pas.

Relancées séparément, `jenga test` répondait **en rouge** et le succès venait du *build*, qui avait raison de réussir. Un défaut d'outil allait être inscrit dans un document partagé.

En diagnostic : une commande, une lecture.

13.3 gdb — déboguer

```
1 jenga gdb --config Debug
2 jenga debug --config Debug      # meme commande
```

Ce qu'il faut retenir

Jenga lance le débogueur sur le bon exécutable, avec la bonne configuration. **Construisez en Debug** : en Release, les symboles sont absents (`symbols(False)`) et vous déboguerez à l'aveugle.

Pour un plantage non interactif, la forme qui donne le plus en une fois :

```
1 gdb --batch -ex "run <args>" -ex "bt full" --args <exe>
```

13.4 watch — reconstruire à chaque enregistrement

```
1 jenga watch
2 jenga w
```

Ce qu'il faut retenir

`watch` surveille vos sources et reconstruit à chaque modification. Il exige `watchdog` — la seule dépendance Python vraiment optionnelle de Jenga.

Il hérite de la décision par dates du chapitre 9, et cette fois à votre avantage : un enregistrement met la date à *maintenant*, donc la recompilation part.

Le piège

Ne laissez pas `watch` tourner pendant une mesure. Il reconstruit au moindre changement — y compris ceux d'un autre outil — et vous mesureriez un binaire différent de celui que vous croyez.

Même règle que pour toute campagne : on ne répare pas la tuyauterie pendant que l'eau coule.

13.5 clean et rebuild

jenga clean – capture réelle, extrait

```
1 Removed ...\\Build\\Obj\\Release-Windows\\Bonjour\\src_main.obj
2 Removed ...\\Build\\Bin\\Release-Windows\\Bonjour\\Bonjour.exe
```

Ce qu'il faut retenir

`clean` énumère ce qu'il supprime, ce qui est plus utile qu'il n'y paraît : la liste montre *quelle* configuration et *quelle* plateforme sont nettoyées.

`jenga clean` sans argument ne nettoie pas nécessairement tout : les autres configurations restent. C'est voulu — et c'est la raison pour laquelle un «j'ai pourtant nettoyé» ne clôt pas une enquête.

Ce qu'il faut retenir

Notez aussi le nom de l'objet : `src_main.obj`. Le chemin de la source est aplati dans le nom — `src/main.cpp` devient `src_main`. C'est ce qui permet à deux fichiers du même nom dans deux dossiers de coexister, et c'est utile à connaître quand on cherche si *son* fichier a bien été compilé (chapitre 9).

Ce chapitre en bref

`run` retrouve l'exécutable et passe vos arguments après `--` ; une application console sans terminal parle dans une console à elle, ce qui explique bien des «programmes muets». `test` rend un code non nul quand il ne trouve rien — «zéro exécuté» n'est pas «zéro en échec» — et le cadre Unitest est fourni par Jenga, pas par votre dépôt. `gdb` veut du Debug, `watch` veut `watchdog` et ne doit pas tourner pendant une mesure. Enfin `clean` énumère, et ne nettoie que la configuration visée.

Questions de cours

1. À quoi sert `--` dans `jenga run` ?
2. Que rend `jenga test` sur un projet sans tests, et pourquoi est-ce le bon comportement ?
3. Quelle différence entre `dutc` et `dute` ?
4. Pourquoi ne faut-il pas laisser `watch` tourner pendant une mesure ?
5. D'où vient le nom `src_main.obj` ?

Exercice pratique

Sur votre projet :

1. ajoutez une suite de tests avec un cas qui passe et un qui échoue ;
2. lancez `jenga test` et relevez le **code de sortie** ;
3. corrigez le cas fautif, relancez, relevez à nouveau ;
4. posez `dute()` dans le workspace et relancez — que se passe-t-il, et le code de sortie change-t-il ?

Le quatrième point est le plus important : c'est celui qui rend un test silencieux.

Exercice de démonstration

Prouvez qu'un enchaînement masque l'origine d'une sortie.

Lancez `jenga test || jenga build` sur un projet *sans* tests, et relevez ce que vous lisez à l'écran.

Puis lancez les deux commandes **séparément** et relevez chaque code de sortie.

Ce que la démonstration établit : la première forme affiche un succès, et rien n'indique qu'il vient de la seconde commande. Vous auriez pu conclure que `test` réussit.

C'est le mécanisme exact qui a fait accuser Jenga à tort sur ce dépôt — et le défaut allait être inscrit dans un document partagé.

Exercice de logique

Une suite de tests « ne s'exécute plus » depuis quelques jours, sans que personne n'ait touché aux tests.

1. Énumérez quatre causes, en distinguant celles qui empêchent la *compilation* de celles qui empêchent l'*exécution*.
2. Pour chacune, dites quel signe la trahit — et laquelle ne laisse *aucun* signe dans la sortie de build.
3. Quel contrôle, ajouté à votre intégration continue, rendrait cette dernière impossible à ignorer?

Livrer : empaqueter, signer, déployer

Un exécutable dans Build/Bin/ n'est pas une livraison. Ce chapitre couvre les quatre commandes qui produisent quelque chose qu'on donne à quelqu'un — et le piège qui a coûté la moitié d'un lot de paquets sur ce dépôt.

14.1 package

```
1 jenga package --platform windows --project MonJeu --output dist/
```

Ce qu'il faut retenir

package CONSTRUIT d'abord. Il ne se contente pas de zipper ce qui traîne : il lance la construction, puis empaquette. La fraîcheur est donc garantie — ce qu'un empaqueteur pur ne peut pas promettre.

C'est une raison de ne pas écrire son propre script de zippage : une seconde façon d'empaqueter finit toujours par diverger de la première.

Le piège qui écrase la moitié d'un lot, en silence

jenga package nomme sa sortie d'après le PROJET, pas d'après la plateforme. Windows produit MonJeu.zip. Web produit MonJeu.zip. Dans le même dossier de sortie, **le second écrase le premier** — code de sortie 0, message «package created», et un artefact de moins.

Le remède : un dossier de sortie *par plateforme*, puis un renommage.



Et ce n'est pas la sortie de la commande qui l'a révélé : c'est l'ouverture de l'archive. On y a trouvé un .wasm là où l'on attendait un .exe. Un empaquetage qui «réussit» cinq fois peut ne laisser que trois artefacts.

Ce qu'il faut retenir

Deux plateformes ne passent pas par package, et il faut le savoir avant d'écrire un script :

Android	l'APK vient de jenga build
HarmonyOS	package exige un hvigor-config.json5 absent

Un script d'empaquetage a donc deux chemins, et c'est délibéré — pas une rustine.

14.2 keygen et sign

```
1 jenga keygen
2 jenga sign
```

Ce qu'il faut retenir

La signature se déclare dans le descripteur, et chaque plateforme a ses champs :

Windows	signingcertificate, signingthumbprint, signingtimestampurl
Android	androidkeystore, androidkeyalias, androidkeystorepass
Apple	signingidentity, signingentitlements
HarmonyOS	harmonycertfile, harmonykeystore, harmonyprofile

signingtimestampurl mérite une phrase : sans horodatage, une signature devient invalide le jour où le certificat expire — même pour un binaire signé bien avant. Avec, elle reste valable.

Le piège

Un APK signé avec la clé de débogage s'installe et ne se publie pas. C'est parfait pour des testeurs internes, et refusé par les magasins.

Le script d'emballage de ce dépôt le dit à chaque exécution : « *APK Release signe avec debug.keystore (OK testeurs internes, pas Play Store)* ».

Dire ce qu'un artefact n'est pas vaut autant que dire ce qu'il est.

14.3 deploy et publish

jenga deploy	installe sur une cible — un téléphone, un appareil
jenga publish	envoie vers une destination de distribution

Ces deux-là sortent vers l'extérieur

Toute commande qui envoie quelque chose hors de votre machine mérite une décision, pas une habitude. Publier un artefact anodin annonce quand même une intention, et ce qui est parti est difficile à reprendre.

Ce livre ne les documente pas plus loin parce qu'elles n'ont pas été exercées ici. *Les décrire d'après leur nom serait exactement ce que le chapitre 12 reproche à un commentaire qui nomme une plateforme.*

14.4 Vérifier un paquet, et non la commande

Ce qu'il faut retenir

« L'empaquetage a réussi » ne prouve rien. Le piège de la section 1 le montre : la commande peut réussir et l'artefact ne pas être celui qu'on croit.

Deux témoins, du plus fort au plus faible :

1. **Lancer ce qui est DANS le paquet.** Extraire l'exécutable et le faire tourner — son banc, s'il en a un. C'est le seul témoin qui porte sur le comportement.
2. **Chercher dans le binaire une chaîne qui n'existe que depuis votre modification.** Un artefact qui la porte a été construit après. Une *date*, elle, se préserve à la copie et ne prouve rien.



Le second témoin exige un contrôle positif : cherchez d'abord une chaîne dont vous savez qu'elle est là. Si vous ne la trouvez pas, votre lecteur ne sait pas lire ce format — et l'absence de la première ne prouve rien du tout.

Ce qu'il faut retenir

Cette méthode a été appliquée sur ce dépôt, sur quinze paquets et cinq plateformes. Résultat : les trois exécutables Windows passent leur banc, et les quatre autres plateformes portent la chaîne témoin.

Et le vérificateur lui-même s'est trompé au premier essai : il comptait le *mot* « ECHEC » et accusait un jeu de deux échecs — qui étaient deux *noms de cas* contenant ce mot. *Compter un mot n'est pas mesurer une chose*, y compris quand c'est vous qui écrivez l'instrument.

14.5 Ce qui accompagne un paquet

Ce qu'il faut retenir

Un dossier de livraison porte plus que des artefacts : il porte de quoi s'en servir. Le lot de ce dépôt contient un `LISEZMOI.md` avec un tableau « fichier / pour qui / comment on s'en sert », et un avertissement en tête :

Build/JeuxPlateau-Release/LISEZMOI.md



1 ## Le Web ne s'ouvre PAS en double-cliquant le `` .html ``

Cette phrase économise plus de messages que toute la documentation technique du lot. Écrivez celle qui correspond à *votre* piège le plus probable.

Ce chapitre en bref

package construit avant d'empaqueter, ce qui garantit la fraîcheur — mais il nomme sa sortie d'après le projet, et deux plateformes s'écrasent en silence dans un même dossier. Android et HarmonyOS passent par `build`, pas par package. La signature se déclare par plateforme, et une clé de débogage donne un paquet installable mais non publiable — il faut le dire. Enfin, un paquet se vérifie en *lançant ce qu'il contient*, pas en lisant la sortie de la commande, et tout témoin par recherche de chaîne exige son contrôle positif.

Questions de cours

1. Que fait `jenga package` avant d'emballer ?
2. D'après quoi nomme-t-il sa sortie, et quelle conséquence ?
3. Quelles plateformes ne passent pas par package ?
4. À quoi sert `signingtimestampurl` ?
5. Citez deux témoins de la fraîcheur d'un paquet, et lequel est le plus fort.

Exercice pratique

Emballer votre projet pour deux plateformes **dans le même dossier**, à dessein.

1. comptez les fichiers produits ;
2. ouvrez celui qui reste et regardez ce qu'il contient ;
3. recommencez avec un dossier par plateforme, puis un renommage ;
4. comparez les deux comptes.

Le point 2 est le cœur de l'exercice : c'est en ouvrant qu'on voit, jamais en lisant la sortie de la commande.

Exercice de démonstration

Prouvez qu'un paquet contient bien vos dernières modifications.

1. ajoutez à votre programme une chaîne unique — la date du jour ;
2. construisez et emballez ;
3. extrayez l'exécutable du paquet et **lancez-le** : il doit afficher la chaîne ;
4. cherchez ensuite cette même chaîne dans le binaire, sans le lancer.

Ce que la démonstration établit : les deux témoins concordent. Faites alors le contraire — emballez *sans* reconstruire après une nouvelle modification — et vérifiez qu'ils sont tous deux capables de dire non.



Un témoin qui n'a jamais dit non n'a rien prouvé.

Exercice de logique

Vous livrez cinq paquets. Un testeur rapporte que « le jeu n'a pas la correction d'hier ».

1. Énumérez quatre causes, depuis « la correction n'est pas dans le dépôt » jusqu'à « le testeur a installé l'ancien ».
2. Pour chacune, dites qui peut la vérifier — vous, ou lui — et avec quoi.
3. Une seule de ces causes se vérifie *sans* le testeur et *sans* reconstruire. Laquelle, et pourquoi commencer par elle ?

Inspecter, générer, brancher son éditeur

Les commandes de ce chapitre ne produisent aucun binaire. Elles répondent à des questions — et ce sont elles qu'on tape quand quelque chose surprend.

15.1 info — ce que Jenga a compris

Ce qu'il faut retenir

C'est la première commande de tout diagnostic, et son intérêt tient en une nuance : elle montre ce que Jenga a *compris*, pas ce que vous croyez avoir écrit. Quatre lignes de sa sortie tranchent des questions différentes :

Ligne	Elle répond à
Entry file	quel workspace a été chargé
Configurations	Debug existe-t-il vraiment
Target OSes	cette plateforme est-elle déclarée
tableau <i>Projects</i>	mon projet est-il inclus
tableau <i>Toolchains</i>	cette chaîne est-elle disponible <i>ici</i>

La dernière ligne est la seule qui décrive la **machine** et non la description. Les autres décrivent votre `.jenga`.

Le piège

« **Projet inconnu** » ne se corrige presque jamais dans le fichier du projet. La bonne question n'est pas « qu'ai-je mal écrit ? » mais « ce fichier est-il inclus ? ». `jenga info` tranche en une seconde : le projet est dans le tableau, ou il n'y est pas. On relit sinon un descripteur parfaitement juste, longtemps.

15.2 compile-flags — les drapeaux réels

jenga compile-flags – capture réelle, extrait

```
1 {"format": "jcdb/1", "compiler": "C:\\msys64\\ucrt64\\bin\\clang++.EXE",
2  "msvc": false, "projects": 1}
```

Ce qu'il faut retenir

C'est le seul outil qui montre ce qu'un filtre a réellement produit. Un `defines` qui n'apparaît pas ici n'a pas été appliqué — et vous savez alors que la condition n'a pas mordu, plutôt que de chercher dans le code.

Notez aussi le *chemin* du compilateur. Sur une machine qui en porte plusieurs, c'est la ligne qui dit lequel travaille — une question qu'on se pose rarement et qui explique des écarts inexpliqués.

Ce qu'il faut retenir

Le format se destine aux diagnostics d'éditeur : c'est ce que consomme un serveur de langage pour savoir avec quels drapeaux votre fichier est compilé.

Sans lui, l'auto-complétion devine — et ses erreurs ne correspondent pas à celles du compilateur, ce qui est la pire des situations : deux avis, dont un faux, sans savoir lequel.

15.3 ide-setup — brancher son éditeur

```
1 jenga ide-setup
2 jenga ide
```

Ce qu'il faut retenir

Elle écrit ce qu'il faut pour que votre éditeur comprenne le projet — `pyrightconfig.json`, le dossier `.jenga-typings/` du chapitre 2, et le fichier de réglages **de chaque éditeur qu'elle détecte**.

`jenga workspace` l'a déjà fait à la création. On la relance après avoir ajouté des projets, ou quand l'auto-complétion s'égare.

«**Détecte**» est le mot qui compte. Jenga cherche une trace, et n'écrit que pour ce qu'il trouve :

ce qu'il cherche	ce qu'il en conclut
<code>.vscode/</code>	VSCode
<code>.nkcode/</code>	NKCode
<code>.idea/</code> ou <code>*.iml</code>	JetBrains
<code>*.sublime-project</code>	Sublime
<code>.nvim.lua</code> ou <code>init.lua</code>	Neovim
<i>rien de tout cela</i>	<code>pyrightconfig.json</code> seul

Sans trace, aucun dossier d'éditeur n'est créé — voir l'encadré du chapitre 3. Pour en obtenir un la première fois, on le demande :

```
1 jenga ide-setup --editor vscode
2 jenga ide-setup --editor lsp
3 jenga ide-setup --info
```

Ce qu'il faut retenir

`--info` **n'écrit rien**. C'est la forme à employer quand on veut seulement savoir ce que Jenga croit voir — et c'est aussi la façon de vérifier qu'il vous a bien reconnu avant de le laisser écrire.

15.4 gen — produire un autre système de construction

```
1 jenga gen --cmake
2 jenga gen --vs
3 jenga gen --makefile
```

Ce qu'il faut retenir

C'est un pont, pas le fonctionnement normal. Il sert quand un outil tiers — un analyseur, un profileur, un éditeur — exige un `CMakeLists.txt` ou une solution Visual Studio.



Le fichier produit est un instantané. Il ne se met pas à jour quand votre `.jenga` change. Deux descriptions coexistent alors, et elles divergeront — c'est mécanique.

Régénérez-le, ne l'éditez pas. Une modification faite dans le fichier généré sera perdue à la prochaine génération, ou pire : survivra et fera diverger les deux systèmes sans que personne le sache.

Ce qu'il faut retenir

Le filtre action: existe pour ce cas :

```
1 with filter("action:gen-cmake"):
2     defines(["GENERATION_CMAKE"])
```

Il permet de décrire quelque chose qui ne vaut que lors d'une génération — par exemple contourner une limite de l'outil cible sans polluer la construction normale.

15.5 Les autres commandes

<code>install</code>	installe dépendances et chaînes d'outils locales
<code>docs</code>	la documentation
<code>examples</code>	les exemples fournis
<code>config</code>	la configuration de Jenga
<code>profile</code>	profilage
<code>bench</code>	mesures
<code>add / file</code>	ajoute des fichiers ou des dépendances à un projet

Ce qu'il faut retenir

Vingt-sept commandes, et quinze alias d'une lettre — `b` pour build, `r` pour run, `i` pour info...

`jenga help` les énumère toutes. C'est la commande à taper avant de supposer qu'une chose n'existe pas : **la liste est plus longue qu'on ne croit**, et l'on réécrit volontiers à la main ce qu'une commande fait déjà.

Les exemples sont livrés avec Jenga

jenga exemples donne accès aux projets de démonstration — [Jenga/Exemples/](#), une trentaine de dossiers du *hello* console au projet multi-plateforme.

Ils sont plus à jour que n'importe quelle documentation, parce qu'ils sont construits par l'intégration continue. Devant un doute sur une fonction, chercher un exemple qui l'emploie coûte moins qu'une lecture d'API.

Ce chapitre en bref

Ces commandes ne construisent rien et répondent aux questions les plus fréquentes. `info` dit ce que Jenga a compris — et sa table des chaînes d'outils est la seule partie qui décrit la machine. `compile-flags` est le seul outil qui montre ce qu'un filtre a réellement produit, et le chemin du compilateur qui travaille. `gen` produit un instantané qui ne se met pas à jour : on le régénère, on ne l'édite pas. Et `jenga help` se tape avant de supposer qu'une chose n'existe pas.

Questions de cours

1. Quelle partie de la sortie de `info` décrit la machine et non votre description ?
2. Comment savoir qu'un filtre n'a pas mordu ?
3. Pourquoi ne faut-il pas éditer un fichier produit par `gen` ?
4. À quoi sert le filtre `action` ?
5. Combien de commandes Jenga offre-t-il, et comment les lister ?

Exercice pratique

Sur votre projet, employez `compile-flags` comme instrument :

1. relevez le compilateur employé et son chemin complet ;
2. ajoutez un `defines` sous un filtre *vrai* et vérifiez qu'il apparaît ;
3. ajoutez-en un sous un filtre *faux* et vérifiez qu'il n'apparaît pas ;
4. ajoutez-en un sous un filtre *mal orthographié* — et constatez que le résultat est le même qu'au point 3.

Le point 4 est celui qui compte : l'outil ne distingue pas une condition fausse d'une condition impossible.

Exercice de démonstration

Prouvez qu'un fichier généré par `gen` ne suit pas votre description.

1. générez un `CMakeLists.txt` ;
2. ajoutez un projet à votre `.jenga` ;
3. relancez `jenga info` — le nouveau projet apparaît ;
4. relisez le `CMakeLists.txt` — il ne l'a pas.

Ce que la démonstration établit : les deux descriptions ont divergé en une seule modification, et rien ne l'a signalé. C'est ce qui rend un fichier généré dangereux dès qu'on le garde.

Exercice de logique

Votre éditeur souligne une erreur que le compilateur ne signale pas.

1. Énumérez trois causes possibles.
2. Laquelle `jenga compile-flags` permet-elle de confirmer, et comment ?
3. Le cas inverse — le compilateur échoue et l'éditeur ne dit rien — a-t-il les mêmes causes ? Justifiez, puis dites laquelle des deux situations est la plus dangereuse et pourquoi.

Ce que cette partie vous laisse

`run` passe vos arguments après `--` ; `test` rend un code non nul quand il ne trouve rien, et «zéro exécuté» n'est pas «zéro en échec». `package` construit avant d'emballer mais nomme sa sortie d'après le projet — deux plateformes s'écrasent en silence. Et les commandes d'inspection sont celles qu'on tape en premier : `info` pour ce que Jenga a compris, `compile-flags` pour ce qu'un filtre a réellement produit.

PARTIE VI

Diagnostiquer

Lire un échec, et retrouver une fonction

Les deux derniers chapitres ne s'apprennent pas : ils se consultent.

Le premier donne l'ordre dans lequel lire un échec — **par coût de vérification, non par probabilité**, ce qui est le bon critère quand on ne sait rien — et la liste des messages qu'on prend pour ce qu'ils ne sont pas.

Le second est une table : vingt-sept commandes, le vocabulaire par famille, les variables de chemin, les préfixes de filtre, les onze chaînes d'outils.

L'exercice pratique du chapitre 16 est le plus utile du livre : provoquer soi-même les cinq échecs classiques et garder leurs messages. Une page de correspondance faite de vos propres captures vaut plus que n'importe quelle documentation — parce qu'elle emploie les mots que *votre* machine affiche.

Quand ça rate : lire ce que Jenga dit

Un échec de construction se lit. Ce chapitre donne l'ordre dans lequel le faire, et les messages qu'on prend pour ce qu'ils ne sont pas.

16.1 Trois étages, trois familles d'échec

Étage	Le message ressemble à	Il faut regarder
chargement	une erreur <i>Python</i>	votre <code>.jenga</code>
compilation	une erreur du <i>compilateur</i>	votre code C++
édition de liens	un <i>symbole</i> manquant	vos <code>links</code> et <code>dependson</code>

Ce qu'il faut retenir

Identifiez l'étage AVANT de chercher. C'est trente secondes, et cela détermine dans quel fichier vous allez passer l'heure suivante.

Le troisième est le plus mal compris. Une erreur d'édition de liens nomme un *symbole* — une fonction, une variable — et ne nomme jamais le module qui la contient. Elle envoie donc chercher dans le code, alors que la cause est dans la description.

16.2 L'échec de chargement

Message	Cause la plus fréquente
<code>SyntaxError</code>	virgule, parenthèse, indentation
<code>NameError</code>	une fonction mal orthographiée
<code>TypeError</code>	une chaîne au lieu d'une liste
<code>FileNotFoundError</code>	un <code>include()</code> vers un fichier absent
<code>No workspace defined</code>	aucun <code>with workspace</code> exécuté
<code>External file not found</code>	un sous-module non peuplé

Ce qu'il faut retenir

--verbose donne la pile complète, et c'est le premier réflexe. Le message dit *quoi*; la pile dit *où*, y compris dans un fichier inclus.

La dernière ligne du tableau accuse le mauvais coupable

`External file not found: Externals/Libs/NKGLad/NKGLad.jenga` ressemble à une erreur de description. C'en est rarement une : **le dossier du sous-module est vide**. Et l'origine est plus retorse encore : dans un *worktree* `git`, `git submodule update --init` ne fait rien et rend 0. Aucune sortie, aucun message. Il faut `--force`.

Tout ce qu'on lance ensuite échoue « à la construction ». **Vérifiez vos sous-modules avant de conclure quoi que ce soit sur le code** — sinon l'instrument accuse le sujet à la place de son propre montage.

16.3 L'échec de compilation

Ce qu'il faut retenir

Jenga relaie le message du compilateur, en nommant le fichier :

Forme réelle

```
1 Compilation Error: NkLudoJeu.cpp
2 .../NkLudoJeu.h:49:42: error: non-virtual member function marked
3 'override' hides virtual member function
```

Le fichier nommé en tête est celui qu'on compilait; l'erreur peut être dans un en-tête. Ici, la compilation de `NkLudoJeu.cpp` échoue sur une ligne de `NkLudoJeu.h` — et les deux se lisent.

Le piège

Le même en-tête peut compiler dans un fichier et échouer dans un autre. Cela paraît absurde et c'est courant : l'en-tête est inclus dans deux contextes différents, avec des macros ou des using différents.

Mesuré sur ce dépôt avec deux types nommés `NkShaderStage` dans deux espaces de noms : un `.cpp` compilait pendant qu'un autre échouait sur le même en-tête.

Dans un en-tête, qualifiez complètement. C'est la seule parade.

Ce qu'il faut retenir

Par défaut, la construction s'arrête au premier échec. Il y a donc exactement une erreur par relevé, et `Failed: 1` ne veut pas dire qu'un seul fichier est cassé.

`--keep-going` continue et donne le compte réel. À employer avant d'estimer un chantier : le premier chiffre est presque toujours faux.

16.4 L'échec d'édition de liens

Symptôme	Ce qu'il faut vérifier
undefined reference to <votre fonction>	le <code>.cpp</code> est-il dans files?
undefined reference to <code>WinMain</code>	le point d'entrée et le genre s'accordent-ils?
cannot find <code>-lX11</code>	la bibliothèque est-elle installée?
framework not found	existe-t-elle sur <i>cette</i> plateforme?
multiple definition of <code>main</code>	deux <code>main</code> — voir <code>excludemainfiles</code>

Ce qu'il faut retenir

La première ligne est la plus fréquente, et la plus mal cherchée. Un symbole qu'on a pourtant écrit est introuvable parce que son fichier n'a jamais été compilé — motif qui ne le prend pas, ou fichier hors du dossier déclaré.

La sortie de construction porte la réponse : Found N source file(s). **Comparez N au nombre de vos fichiers** avant de chercher ailleurs.

Le piège

Une erreur d'édition de liens ne nomme aucun de vos fichiers de description. undefined reference to WinMain ne mentionne ni votre .jenga, ni le mot windowedapp — et l'on cherche partout sauf là.

Quand un message ne nomme aucun de vos fichiers, la cause est presque toujours dans la description.

16.5 L'ordre de diagnostic

Question	Commande
1 le bon workspace est-il chargé ?	jenga info (ligne <i>Entry file</i>)
2 mon projet est-il dedans ?	jenga info (tableau)
3 mes fichiers sont-ils trouvés ?	Found N source file(s)
4 mes drapeaux sont-ils appliqués ?	jenga compile-flags
5 mon fichier a-t-il été compilé ?	ligne Compiled:
6 le binaire est-il frais ?	jenga rebuild

Ce qu'il faut retenir

Cet ordre est celui du COÛT, pas celui de la probabilité. Les quatre premières questions se répondent sans rien construire; la sixième coûte une construction complète.

Quand on ne sait rien, ordonner par coût est le bon critère — ordonner par probabilité suppose qu'on sait déjà quelque chose.

16.6 Les messages qui ne sont pas des erreurs

Message	Ce que c'est
No test projects found.	normal si vous n'avez pas de tests — mais code 1
Application console sans terminal	une commodité de run
APK signe avec debug.keystore	un avertissement de <i>destination</i>
avertissement de version embarquée	NKCode vous prévient d'une divergence

Le piège

Un avertissement noyé au milieu de mille lignes n'a pas été émis. Mesuré : l'avertissement «keystore not found» de Jenga apparaît au milieu du journal de construction et n'est *pas répété* après le SUCCESS final.

Résultat : un APK sans signature, un adb install qui refuse, et une enquête qui commence par le téléphone.

Quand vous écrivez un outil, répétez à la fin ce qui compte. Un message qui n'est dit qu'une fois, au milieu, est un message qu'on n'a pas dit.

Ce chapitre en bref

Trois étages d'échec — chargement, compilation, édition de liens — et identifier lequel avant de chercher détermine où l'on passe l'heure suivante. Le troisième est le plus mal compris : il nomme un symbole et jamais la description, qui est pourtant sa cause habituelle. Un sous-module vide se présente comme une erreur de description ; la construction s'arrête au premier échec, donc le premier compte est faux. Diagnostiquez dans l'ordre du *coût*, pas de la probabilité — quatre des six questions se répondent sans rien construire. Et un avertissement dit une seule fois, au milieu d'un journal, n'a pas été dit.

Questions de cours

1. Comment reconnaît-on l'étage d'un échec ?
2. Que faire devant `External file not found` sur un `.jenga` de dépendance ?
3. Pourquoi `Failed: 1` est-il presque toujours faux ?
4. Quelle ligne consulter devant un `undefined reference` sur une fonction que vous avez écrite ?
5. Pourquoi ordonner un diagnostic par coût plutôt que par probabilité ?

Exercice pratique

Provoquez délibérément les cinq échecs, et **gardez le message de chacun** :

1. une virgule manquante dans le `.jenga` ;
2. une fonction `Jenga` mal orthographiée ;
3. un `files()` qui ne trouve rien ;
4. un `links` vers une bibliothèque inexistante ;
5. un `windowedapp()` sur un programme qui n'a qu'un `main` classique.

Constituez-vous une page de correspondance message → cause. **C'est le document le plus utile que vous écrirez sur Jenga**, et il ne vaut que s'il est fait de vos propres captures.

Exercice de démonstration

Prouvez qu'une erreur d'édition de liens ne désigne pas sa cause.

Retirez un fichier de votre `files()` — sans le supprimer du disque — et construisez.

Ce que la démonstration doit établir, en trois relevés :

1. l'erreur nomme un *symbole*, pas un fichier de description ;
2. elle ne contient aucune occurrence du mot `files` ni du nom de votre `.jenga` ;
3. la ligne `Found N source file(s)`, elle, a changé — et c'était le seul indice, dix lignes plus haut.

Le troisième point est la leçon : l'information était là, avant l'erreur.

Exercice de logique

Une construction qui passait hier échoue aujourd'hui, sans qu'aucun fichier source n'ait changé.

1. Énumérez cinq causes possibles, dont aucune n'est dans le code C++.
2. Rangez-les par coût de vérification.
3. Deux d'entre elles ne laissent *aucune trace* dans la sortie de construction. Lesquelles, et par quoi les prendriez-vous ?

Référence : les commandes et le vocabulaire

Ce chapitre est une table. Il se consulte, il ne se lit pas — et il est relevé sur la version 2.4.0.

17.1 Les vingt-sept commandes

Commande	Alias	Ce qu'elle fait
build	b	compile le workspace ou un projet
run	r	exécute un projet
test	t	lance les tests unitaires
gdb	g, debug	lance le débogueur
clean	c	supprime objets et artefacts
rebuild	—	clean puis build
watch	w	reconstruit à chaque modification
info	i	ce que Jenga a compris
gen	—	produit CMake, Visual Studio, Makefile
workspace	init	crée un espace de travail
project	create	crée un projet
file	add	ajoute fichiers ou dépendances
install	—	installe dépendances et chaînes locales
keygen	k	génère une clé de signature
sign	s	signe un artefact
docs	d	la documentation
package	—	construit puis empaquette
deploy	—	installe sur une cible
publish	—	envoie vers une distribution
profile	—	profilage
bench	—	mesures
config	—	configuration de Jenga
examples	e	les exemples fournis
ide-setup	ide	configure l'éditeur
compile-flags	—	les drapeaux réels, en JSON
help	h	la liste

Ce qu'il faut retenir

Les alias ne sont pas tous d'une lettre, et deux d'entre eux sont des *synonymes* plutôt que des raccourcis : `debug` pour `gdb`, `ide` pour `ide-setup`. Et deux paires sont réversibles : `workspace/init`, `project/create`, `file/add`. Employez le même dans une équipe : deux orthographes rendent une recherche dans les scripts incomplète.

17.2 Le vocabulaire, par famille

Structure

<code>workspace(nom)</code>	le bloc racine
<code>project(nom)</code>	un artefact
<code>filter(expr)</code>	une condition
<code>include(chemin)</code>	exécute un autre .jenga en ligne
<code>unittest()</code>	configure le cadre de test
<code>toolchain(...)</code>	déclare une chaîne d'outils

Genre et langage

<code>consoleapp()</code>	<code>windowedapp()</code>	exécutables
<code>staticlib()</code>	<code>sharedlib()</code>	bibliothèques
<code>testsuite()</code>		suite de tests
<code>kind(k)</code>		la forme générique
<code>language(l)</code>		"C" ou "C++"
<code>cppdialect</code>	<code>cdialect</code>	la norme

Fichiers et chemins

<code>location</code>	où vit le projet
<code>files</code> <code>excludefiles</code> <code>excludemainfiles</code>	les sources
<code>includedirs</code> <code>externalincludedirs</code> <code>sysincludedirs</code>	en-têtes
<code>libdirs</code> <code>syslibdirs</code>	où chercher les bibliothèques
<code>objdir</code> <code>targetdir</code> <code>bindir</code> <code>targetname</code>	les sorties
<code>dependfiles</code> <code>embedresources</code>	fichiers copiés / intégrés

Dépendances et drapeaux

dependson removedependson	entre projets
links removedlinks	bibliothèques
frameworks framework	Apple
defines removedefines	définitions
optimize symbols runtime	réglages de compilation
cflags cxxflags ldflags arflags	drapeaux bruts
prelink postlink	actions autour du lien

Ce qu'il faut retenir

Les ***flags** sont une porte de sortie, pas un outil courant. Ils passent une chaîne au compilateur sans que Jenga la comprenne — donc sans qu'il puisse l'adapter à une autre chaîne d'outils.

Un drapeau brut est **lié à un compilateur**. Employez-le quand rien d'autre ne convient, dans un filtre toolset:, et écrivez pourquoi.

Réglages courts

debug() release() reldebinfo() minsize() rel()	profils
debuggdb() debuglldb() debugcodeview()	format de symboles
sanitizeaddress() sanitizethread() ...	analyseurs dynamiques
coveragecov() coveragevs()	couverture
profilegprof() profilevs() profileinstruments()	profilage
pic() pie() nostdlib() nostdinc()	génération de code

Métadonnées et plateformes

apppublisher appversion licensefile appicon	communes
android* (une trentaine)	Android
ios* tvos* watchos* visionos*	Apple
harmony* (une quinzaine)	HarmonyOS
emscripten*	Web
xbox* gdkpath	Xbox
signing*	signature
networkenabled firewallrule	réseau

Ce qu'il faut retenir

Le vocabulaire complet compte **242 fonctions publiques**, relevées dans [Jenga/Core/Api.py](#). Les tables ci-dessus en donnent la charpente.

Devant un besoin précis, cherchez par préfixe : android, ios, harmony, emscripten, signing. La convention de nommage est régulière, et c'est ce qui rend l'API cherchable sans documentation.

17.3 Les variables de chemin

<code>%{wks.location}</code>	le dossier du workspace
<code>%{prj.name}</code>	le nom du projet
<code>%{prj.location}</code>	le dossier du projet
<code>%{cfg.buildcfg}</code>	Debug ou Release
<code>%{cfg.system}</code>	le système visé

17.4 Les préfixes de filtre

<code>system:</code>	le système visé
<code>config: configuration: configurations: cfg:</code>	la configuration
<code>arch: architecture:</code>	l'architecture
<code>platform: platforms:</code>	système + architecture
<code>action:</code>	la commande en cours
<code>options: option:</code>	une option déclarée
<code>kind:</code>	le genre du projet
<code>language:</code>	le langage
<code>toolset:</code>	la famille de compilateur

17.5 Les onze chaînes d'outils

Nom	Famille	Cible
<code>host-apple-clang</code>	apple-clang	la machine
<code>host-clang</code>	clang	la machine
<code>host-gcc</code>	gcc	la machine
<code>clang-mingw</code>	clang	Windows
<code>mingw</code>	gcc	Windows
<code>msvc</code>	msvc	Windows
<code>gcc-cross-linux</code>	gcc	Linux
<code>clang-cross-linux</code>	clang	Linux
<code>emscripten</code>	emscripten	Web
<code>android-ndk</code>	android-ndk	Android
<code>ohos-ndk</code>	clang	HarmonyOS

Ce qu'il faut retenir

Déclarées ≠ disponibles. `jenga info` ne montre que celles qu'il a trouvées sur la machine. Un descripteur qui exige une chaîne absente échoue au chargement — et le message parle de la chaîne, pas du SDK manquant.

Ce chapitre en bref

Vingt-sept commandes, quinze alias, 242 fonctions de vocabulaire, neuf préfixes de filtre, onze chaînes d'outils. Les tables de ce chapitre en donnent la charpente; le reste se cherche par préfixe, la convention de nommage étant régulière. Retenez deux choses : les drapeaux bruts sont une porte de sortie liée à un compilateur, et une chaîne d'outils déclarée n'est pas une chaîne disponible.

Questions de cours

1. Quelle commande fait `clean` puis `build`?
2. Quels sont les deux alias qui sont des synonymes et non des raccourcis?
3. Pourquoi un drapeau brut est-il moins portable qu'un réglage nommé?
4. Que valent `%{cfg.buildcfg}` et `%{cfg.system}`?
5. Quelle différence entre une chaîne d'outils déclarée et disponible?

Exercice pratique

Prenez trois besoins et trouvez la fonction, *sans* lire ce chapitre :

1. embarquer une police dans le binaire;
2. interdire l'usage de la bibliothèque standard;
3. déclarer les permissions Android.

jenga `help`, les exemples fournis et la recherche par préfixe suffisent. Chronométrez : si l'un vous prend plus de deux minutes, notez lequel — c'est un manque de la convention de nommage, et il vaut d'être signalé.

Exercice de démonstration

Prouvez qu'une chaîne d'outils déclarée peut être indisponible.

1. relevez la liste que jenga `info` affiche sur votre machine;
2. comparez-la aux onze du tableau ci-dessus;
3. écrivez un `usetoolchain` vers une chaîne absente de votre liste et lancez jenga `build`.

Ce que la démonstration doit établir : le message d'échec parle de la *chaîne*, pas du SDK qui lui manque. Relevez-le mot pour mot — c'est celui que vous reverrez sur une machine neuve, et vous saurez alors quoi installer plutôt que quoi corriger.

Exercice de logique

Vous héritez d'un descripteur qui emploie `cxxflags(["-O3"])` sous un filtre `config:Release`, alors que `optimize("Speed")` existe.

1. Citez deux raisons légitimes de ce choix.
2. Citez deux problèmes qu'il crée, dont un qui ne se verra que sur une autre plateforme.
3. Que feriez-vous — et qu'écririez-vous à côté, quel que soit votre choix?

Ce que cette partie vous laisse

Un échec appartient à l'un des trois étages — chargement, compilation, édition de liens — et l'identifier avant de chercher détermine où l'on passe l'heure suivante. Le troisième nomme un symbole et jamais la description, qui est pourtant sa cause habituelle. Diagnostiquez dans l'ordre du coût : quatre des six questions se répondent sans rien construire. Et devant un besoin précis, cherchez le vocabulaire par préfixe — la convention de nommage est régulière, ce qui rend l'API cherchable sans documentation.